

ورشة عملية — مشروع واحد من الصفر حتى النشر

ابن مشروع Fullstack خطوة بخطوة

NestJS · Prisma · PostgreSQL · Next.js 15 · TypeScript

مشروع واحد اسمه `mini-shop` تبنيه بيدك من مجلد فارغ إلى تطبيق منشور على الإنترنت. كل خطوة فيها: **الهدف**، ثم الأمر أو الكود، ثم الشرح، ثم كيف تجربها وما النتيجة المتوقعة.

يغطي الأساسيات الحقيقية التي تُسأل عنها في المقابلات: **مصادقة كاملة** بـ JWT وأدوار وصلاحيات، **REST API كامل** بترقيم وفلتر وبحث وعلاقات ورفع ملفات، **تعدد اللغات** على ثلاث طبقات (رسائل الـ API، محتوى قاعدة البيانات، واجهة المستخدم)، إضافة إلى التحقق والأخطاء والتوثيق والاختبارات والنشر.

لا يوجد في هذا الكتاب سطر واحد لا تكتبه بنفسك.

اقرأ الخطوة → نَقِّذها على جهازك → شغّل التجربة → قارن النتيجة → انتقل للتالية.

مبني على إصدارات: Node 22+ · NestJS 11 · Prisma 6 · Next.js 15 · PostgreSQL 16

محتويات الورشة

عشر مراحل — ثمان وسبعون خطوة مرقّمة بالتسلسل

- قبل أن نبدأ
 - ما الذي سنبنيه · مخطّط النظام · شجرة المشروع النهائية · كيف تقرأ هذا الكتاب
- ١ البيئة وقاعدة البيانات
 - فحص الأدوات · مجلّد المشروع · Postgres عبر Docker · التحقق من الاتصال · الخطوات ١-٥
- ٢ إنشاء ال API وضبطه
 - nest new · جولة على الملفات · ValidationPipe · CORS · main.ts · متغيّرات البيئة ٦-١٢
- ٣ قاعدة البيانات والهجرات
 - Prisma · المخطّط · العلاقات · أوّل PrismaService · migration · بذور البيانات ١٣-١٩
- ٤ أوّل مورد REST كامل
 - nest g resource · DTO · التحقق · الخدمة · المتحكّم · الأخطاء · شكل الاستجابة ٢٠-٢٧
- ٥ REST بمعناه الكامل
 - الترقيم · الفلترة والبحث والترتيب · العلاقات و N+1 · رفع الصور · ٢٨-٣٥
- ٦ المصادقة والصلاحيات
 - تجزئة كلمة السر · تسجيل ودخول · refresh token · JWT · الحارس العام · الأدوار ٣٦-٤٦
- ٧ تعدّد اللغات في ال API
 - الطبقات الثلاث · nestjs-i18n · ترجمة رسائل التحقق · محتوى مترجم في قاعدة البيانات ٤٧-٥٢
- ٨ الواجهة بـ Next.js
 - create-next-app · next-intl · مسار اللغة · RTL · طبقة الاتصال · صفحات المنتجات ٥٣-٦٢
- ٩ الدخول ولوحة الإدارة
 - Server Action · كوكبي httpOnly · حماية المسارات · نموذج المنتج · إعادة التحقق ٦٣-٧٠
- ١٠ الجاهزية والنشر
 - Swagger · السجّلات · الحدّ من المحاولات · الاختبارات · Docker · النشر ٧١-٧٨
- ★ الخاتمة
 - قائمة تحقق نهائية · ماذا تضيف بعد ذلك · أسئلة مقابلة على هذا المشروع بالذات

قبل أن نبدأ

خمس دقائق قراءة توفر عليك ساعات من الحيرة لاحقًا.

ما الذي سنبنيه

مشروع اسمه `mini-shop`: متجر مصغّر بواجهة عامة ولوحة إدارة. اخترته لأنّه يجبرنا على لمس كل المفاهيم التي تُسأل عنها في المقابلات، دون أن يتضح:

الميزة	ما الذي تعلّمك إياه
تسجيل ودخول	تجزئة كلمات السر، JWT، الرمز المُحدّث (<code>refresh token</code>)، تخزين الجلسة في المتصفح
أدوار (<code>USER / ADMIN</code>)	الخُرّاس (<code>Guards</code>)، المزخرفات المخصّصة، الفرق بين <code>401</code> و <code>403</code>
أصناف ومنتجات	مورد <code>REST</code> كامل: إنشاء، قراءة، تعديل، حذف، مع رموز حالة صحيحة
قوائم كبيرة	الترقيم (<code>pagination</code>)، الفلترة، البحث، الترتيب، والحدّ من الحمل
علاقات	واحد-لـكثير، الربط في <code>Prisma</code> ، ومشكلة <code>N+1</code> وكيف تتجنّبها
صورة للمنتج	رفع الملفات، التحققّ من النوع والحجم، تقديم الملفات الساكنة
عربي وإنجليزي	تعدّد اللغات على ثلاث طبقات مختلفة تمامًا — وهذا ما يخلط على أغلب المرشّحين
لوحة إدارة	النماذج، الإرسال من الخادم (<code>Server Actions</code>)، إعادة التحققّ من الذاكرة المؤقتة

مخطّط النظام

BROWSER	NEXT.JS (web)	NESTJS (api)	POSTGRES
-----	-----	-----	-----
/ar/products	-----> Server Component renders HTML	--fetch--> GET /api/products ProductsController	----> SELECT ... rows
	<--HTML----	 ProductsService	
login form	-----> Server Action sets httpOnly cookie	--fetch--> POST /api/auth/login AuthService (bcrypt + JWT)	
/ar/admin/...	-----> middleware checks cookie then Client Component	--fetch (Bearer)--> protected routes	

- المتصفح لا يتكلّم مع الـ API مباشرة في الصفحات العامة — خادم `Next` يجلب البيانات ويرسل `HTML` جاهزًا.
- خادم `Next` هو أيضًا حارس البوابة: يحتفظ برمز الدخول في كوكي `httpOnly` لا يستطيع `JavaScript` قراءته.
- الـ API منفصل تمامًا. تقدر تشغله وحده وتختبره بـ `curl` أو `Postman` بلا واجهة إطلاقًا.
- قاعدة البيانات تعمل داخل حاوية `Docker`، فلا تلوّث جهازك بتثبيت دائم.

لماذا هذا الفصل مهم في المقابلة: السؤال «لماذا تفصل الواجهة عن الـ API؟» يتكرّر كثيرًا. الجواب العملي: لأنّ الـ API نفسه يخدم لاحقًا تطبيق موبايل أو لوحة تحكم ثانية أو تكاملًا مع طرف ثالث، ولأنّ فريقين يقدران يشتغلان بالتوازي ما دام العقد (`the contract`) بينهما ثابتًا.

شجرة المشروع عند النهاية

هذه صورة المستقبل. لا تُنشئ شيئًا الآن — كل مجلد هنا سيُولد في خطوة محدّدة لاحقًا.

```
mini-shop/
├─ docker-compose.yml      قاعدة البيانات للتطوير
├─ api/                    الخادم — NestJS
│ │ └─ prisma/
│ │ │ └─ schema.prisma    تعريف الجداول والعلاقات
│ │ │ └─ migrations/     تاريخ تغييرات قاعدة البيانات
│ │ │ └─ seed.ts          بيانات ابتدائية
│ │ └─ src/
│ │ │ └─ main.ts          نقطة الإقلاع
│ │ │ └─ app.module.ts    الوحدة الجذر
│ │ │ └─ config/          متغيّرات البيئة والتحقّق منها
│ │ │ └─ prisma/          PrismaService + PrismaModule
│ │ │ └─ common/          فلاتر، مزخرفات، أنواع مشتركة
│ │ │ └─ auth/            التسجيل والدخول والحُزاس
│ │ │ └─ users/
│ │ │ └─ categories/
│ │ │ └─ products/
│ │ │ └─ i18n/            API لرسائل الـ ar.json و en.json
│ │ └─ test/              اختبارات e2e
│ └─ uploads/             صور المنتجات
├─ .env
├─ Dockerfile
└─ web/                   الواجهة — Next.js
    ├─ messages/{ar,en}.json  نصوص الواجهة
    └─ src/
        ├─ middleware.ts     توجيه اللغة وحماية المسارات
        ├─ i18n/
        ├─ lib/api.ts        طبقة واحدة للاتصال بالخادم
        └─ app/[locale]/
            ├─ layout.tsx
            ├─ page.tsx
            └─ products/
                ├─ login/
                └─ admin/
└─ .env.local
```

كيف تقرأ هذا الكتاب

كل خطوة مبنية على نفس القالب. تعرّف عليه مرّة واحدة وستطير في الباقي:

العنصر	ماذا يعني
عنوان الخطوة	رقمها وهدفها في سطر واحد.
الشرح	لماذا نفعل هذا، وما البديل الذي رفضناه ولماذا.
صندوق رمادي محدّد داكن	أمر تكتبه في الطرفيّة (terminal).
صندوق بنفسجي بعنوان ملف	محتوى ملف — الاسم فوق الصندوق هو مسار الملف بالضبط.
صندوق أخضر «جذبها»	كيف تتأكّد أنّ الخطوة نجحت، وما النتيجة المتوقّعة حرفيّاً.
صندوق أحمر «انتبه»	خطأ شائع سيوقعك لو تجاهلته.
صندوق أزرق «تمرين»	عمل إضافي بلا حلّ مكتوب — هنا يحصل التعلّم الحقيقي.

قاعدة الورشة

اكتب الكود بيدك. لا تنسخ وتلصق. النسخ يعطيك مشروّعاً يعمل ورأساً فارغاً؛ الكتابة باليد تجعلك تخطئ، والخطأ ثم إصلاحه هو ما يبقى في ذاكرتك يوم المقابلة.

إذا توقّف شيء عن العمل

لا تقفز للخطوة التالية. تسعون بالمئة من الأعطال في هذه الورشة أحد ثلاثة: الخادم غير مُعاد التشغيل بعد تعديل `.env`، أو حاوية قاعدة البيانات متوقّفة (`docker ps`)، أو نسيت تسجيل الوحدة الجديدة في `app.module.ts`. اقرأ رسالة الخطأ كاملة — رسائل `Nest` و `Prisma` تقول لك المشكلة بصراحة غالباً.

البيئة وقاعدة البيانات

قبل أي كود: أدوات صحيحة، مجلد منظم، وقاعدة بيانات تعمل ومُختبرة.

خطوة ١ تأكد من الأدوات

Node و npm و git — ولا شيء غيرها مطلوب تثبيته يدويًا.

```
node -v
npm -v
git --version
```

يجب أن ترى شيئًا قريبًا من هذا:

```
v22.14.0
10.9.2
git version 2.45.0
```

المهم هو الرقم الأول من **Node**: لازم 20 أو أعلى، والأفضل 22 أو أحدث. 11 NestJS و 15 Next.js يرفضان الإصدارات الأقدم. إذا كان عندك رقم أقل، نزل الإصدار LTS من nodejs.org وأعد فتح الطرفية.

مقارنة سريعة بما تعرفه: node يقابل php (المفسر)، و npm يقابل composer (مدير الحزم)، و package.json يقابل composer.json، و node_modules يقابل vendor/. الفرق العملي الوحيد المزعج: node_modules أضخم بكثير — هذا طبيعي، لا تقلق.

خطوة ٢ أنشئ مجلد المشروع

مستودع واحد يحوي الخادم والواجهة جنبًا إلى جنب.

```
mkdir mini-shop
cd mini-shop
git init
```

سنضع `api/` و `web/` داخل هذا المجلد. هذا يسمى `monorepo` بسيط: مستودع واحد فيه مشروعان مستقلان، لكل واحد `package.json` خاص به. لسنا بحاجة إلى أدوات `monorepo` متقدمة مثل `Nx` أو `Turborepo` في هذا الحجم — إضافتها الآن تعقيد بلا مقابل.

وقبل أي شيء، ملف تجاهل واحد على مستوى الجذر:

mini-shop/.gitignore

```
node_modules/  
dist/  
.next/  
.env  
.env.local  
*.log  
uploads/
```

انتبه

`.env` يجب ألا يدخل `git` أبدًا. هذا أول ما يفحصه المُقايِل حين ينظر إلى مستودعك، وتسريب مفتاح في تاريخ الـ `commits` لا يُحلّ بحذف الملف لاحقًا. أنشئ `.gitignore` قبل أول `commit`، لا بعده.

شغل PostgreSQL عبر Docker خطوة ٣

قاعدة بيانات حقيقية بلا تثبيت دائم على جهازك.

نحتاج قاعدة بيانات. الخيار الأسهل للتجربة هو `SQLite` (ملف واحد، صفر إعداد)، لكننا سنستخدم `PostgreSQL` لسببين: هو ما تطلبه الوظائف فعليًا، وبعض الميزات التي سنستعملها لاحقًا (البحث غير الحساس لحالة الأحرف، الأنواع المحدودة `enum`، الفهارس المركبة) تتصرّف في `SQLite` بشكل مختلف فتتعلّم شيئًا خاطئًا.

ثبّت `Docker Desktop` إن لم يكن موجودًا، ثم:

mini-shop/docker-compose.yml

```
services:  
  db:  
    image: postgres:16-alpine  
    container_name: minishop-db  
    restart: unless-stopped  
    environment:  
      POSTGRES_USER: shop  
      POSTGRES_PASSWORD: shop_dev_pass  
      POSTGRES_DB: minishop  
    ports:  
      - "5433:5432" 5433 على جهازك ← 5432 داخل الحاوية  
    volumes:  
      - dbdata:/var/lib/postgresql/data  
  
volumes:  
  dbdata:
```

```
docker compose up -d
```

`-d` تعني «في الخلفية». استخدمنا المنفذ `5433` بدل `5432` لأنّ كثيرين لديهم `Postgres` مثبت أصلاً على المنفذ الافتراضي، فيحدث تعارض محيّر. و `volumes` تعني أنّ بياناتك تنجو من إيقاف الحاوية — بدونها تفقد كل شيء عند كل إعادة تشغيل.

خطوة ٤ تحقق أن قاعدة البيانات تستجيب

لا تبين فوق أساس لم تختبره.

جربها الآن

```
docker ps
```

يجب أن ترى سطرًا فيه minishop-db وحالته Up. ثم ادخل إلى القاعدة نفسها:

```
docker exec -it minishop-db psql -U shop -d minishop -c "SELECT version();" 
```

النتيجة المتوقعة سطر يبدأ بـ PostgreSQL 16.x on x86_64... لو ظهر هذا، القاعدة جاهزة.

إن فشل

port is already allocated ← المنفذ مشغول، غيّر 5433 إلى 5434 وأعد docker compose up -d.
Cannot connect to the Docker daemon ← Docker Desktop غير مُشغّل، افتحه وانتظر حتى تستقر الأيقونة.
database system is starting up ← انتظر بضع ثوانٍ وأعد المحاولة، الحاوية ما زالت تُقْلَع.

أوامر تحتاجها كل يوم: docker compose up -d تشغيل · docker compose stop إيقاف مع الاحتفاظ بالبيانات · docker compose down -v حذف كل شيء بما فيه البيانات (استعملها للبدء من الصفر) · docker compose logs -f db متابعة سجلات القاعدة.

خطوة ٥ جهّز محرّرك

إضافتان توقّران عليك نصف الأخطاء الإملائية.

في VS Code ثبّت هاتين الإضافتين:

- **Prisma** — تلوين وإكمال تلقائي وتنسيق لملف schema.prisma.
 - **ESLint** — يُظهر أخطاء الأسلوب مباشرة في المحرّر بدل انتظار الأمر.
- وإضافة اختيارية مفيدة جدًّا: **REST Client**، تسمح لك بكتابة الطلبات في ملف http. وتشغيلها بنقرة، بديلًا خفيًّا عن Postman. سنستخدمها في التجارب.

تمرين المرحلة ١

1. أوقف الحاوية بـ docker compose stop ثم أعد تشغيلها. تحقق أنّ docker ps يُظهرها مجدّدًا.
2. غيّر اسم قاعدة البيانات في docker-compose.yml إلى shopdb، ثم شغّل docker compose up -d. هل تغيّر شيء؟ لماذا لا؟ (تلميح: الـ volume القديم ما زال موجودًا — جرّب down -v ثم up -d). ثم أعد الاسم إلى minishop.
3. اعمل أوّل commit: git add . && git commit -m "chore: project skeleton". افحص بـ git status أنّ لا شيء من المتجاهلات دخل.

إنشاء ال API وضبطه

مشروع NestJS يعمل، مفهوم ملفاً ملفاً، ومضبوط قبل أن نكتب أي منطق.

خطوة ٦ أنشئ مشروع NestJS

أمر واحد يولّد الهيكل كاملاً.

من داخل `mini-shop/`:

```
npx @nestjs/cli@latest new api --package-manager npm --strict
```

سيسألك عن مدير الحزم إن لم تمرّر الخيار — اختر `npm --strict` — يفقّل الوضع الصارم في `TypeScript` منذ اللحظة الأولى؛ شغلّه الآن لأنّ تفعيله بعد مئة ملف يعني خطأ دفعة واحدة.

ثم:

```
cd api
npm run start:dev
```

جربها الآن

افتح `http://localhost:3000` في المتصفح. المتوقّع: كلمة `Hello World!` على الشاشة، وفي الطرفيّة سطور خضراء تنتهي بـ:

```
[Nest] LOG [NestApplication] Nest application successfully started
```

اترك هذه الطرفيّة مفتوحة طوال الورشة — `start:dev` يراقب الملفات ويعيد التشغيل تلقائيًا عند كل حفظ.

خطوة ٧ افهم ما وُلّد

جولة على كل ملف: ما هو، وما مقابله في `Laravel`.

api/	
├ src/	
├── main.ts	نقطة الإقلاع ← <code>public/index.php</code>
├── app.module.ts	الوحدة الجذر ← <code>AppServiceProvider + routes</code>
├── app.controller.ts	متحكّم ← <code>app/Http/Controllers</code>
├── app.service.ts	خدمة ← <code>app/Services</code>
└ app.controller.spec.ts	اختبار وحدة ← <code>tests/Unit</code>
└ test/	اختبارات ← <code>tests/Feature</code> ← <code>e2e</code>
├ nest-cli.json	إعدادات مولّد الشيفرة
├ tsconfig.json	إعدادات <code>TypeScript</code>
├ eslint.config.mjs	قواعد جودة الشيفرة
└ package.json	← <code>composer.json</code>

افتح `app.module.ts` وانظر إليه جيّدًا، فهو مفتاح فهم `Nest` كلّّه:

api/src/app.module.ts

```
import { Module } from "@nestjs/common";
import { AppController } from "../app.controller";
import { AppService } from "../app.service";

@Module({
  imports: [], // وحدات أخرى نعتمد عليها
  controllers: [AppController], // من يستقبل طلبات HTTP
  providers: [AppService], // من يُحقن في غيره (الخدمات)
})
export class AppModule {}
```

الوحدة (Module): صندوق يجمع متحكمات وخدمات تخصّ ميزة واحدة، ويُعلن ما الذي يستورده من غيره وما الذي يُصدّره لغيره. أقرب شيء في **Laravel** هو الـ **Service Provider**، لكن الفرق الجوهرى أنّ الوحدة في **Nest** **تحدّد نطاق الحقن**: خدمة غير مُصدّرة من وحدتها لا يمكن حقنها في وحدة أخرى — والمحاولة تُفشل بالإقلاع برسالة واضحة.

وافتح المتحكم:

api/src/app.controller.ts

```
@Controller() // بادئة المسار (فارغة هنا)
export class AppController {
  constructor(private readonly appService: AppService) {}
  // هذا السطر وحده هو حقن التبعية — لا new ولا container يدوي

  @Get() // ← Route::get('/')
  getHello(): string {
    return this.appService.getHello();
  }
}
```

الفرق الذهني الأكبر عن **Laravel**

في **Laravel** المسارات تُكتب في ملف منفصل (**routes/api.php**) وتُشير إلى المتحكمات. في **Nest** لا يوجد ملف مسارات إطلاقاً — المسار يُعلن فوق الدالة نفسها بمزخرف **@Get()** أو **@Post()**. لتعرف مسارات ميزة، اذهب إلى متحكمها مباشرة.

خطوة ٨ اضبط **main.ts**

بادئة، CORS، تحقّق عام، وإطفاء لطيف.

هذا الملف هو ما يقابل **bootstrap/app.php** عندك: أربعة أسطر هنا توقّر عليك مشاكل ستلاحقك طوال المشروع.

```
import { NestFactory } from "@nestjs/core";
import { ValidationPipe } from "@nestjs/common";
import { AppModule } from "../app.module";

async function bootstrap() {
  const app = await NestFactory.create(AppModule);

  كل المسارات تصير تحت /api — يسهّل النشر خلف بروكسي واحد
  app.setGlobalPrefix("api");

  السماح لواجهة Next (منفذ 3001) بالاتصال
  app.enableCors({
    origin: ["http://localhost:3001"],
    credentials: true,
  });

  التحقق التلقائي من كل جسم طلب — تفصيله في المرحلة ٤
  app.useGlobalPipes(
    new ValidationPipe({
      whitelist: true,          احذف أي حقل غير معرّف في الـ DTO
      forbidNonWhitelisted: true, بل ارفض الطلب وأخبره بالحقل الزائد
      transform: true,          حوّل النصوص إلى أنواعها (ID نص ← رقم)
    }),
  );

  أغلق الاتصالات بنظافة عند الإيقاف
  app.enableShutdownHooks();

  const port = process.env.PORT ?? 3000;
  await app.listen(port);
  console.log(`API on http://localhost:${port}/api`);
}
bootstrap();
```

جربها الآن

احفظ الملف. ستُعاد التشغيل تلقائيًا. الآن `http://localhost:3000` يعطي 404، و `http://localhost:3000/api` يعطي Hello World! هذا يثبت أنّ `setGlobalPrefix` فعّال.

CORS: الخطأ الذي يضيّع ساعة على كل مبتدئ

الـ API يعمل تمامًا في Postman ثم يفشل من المتصفّح برسالة `blocked by CORS policy`. السبب أنّ Postman ليس متصفّحًا ولا يطبّق سياسة الأصل. الحل ليس تعطيل الأمان في المتصفّح، بل السماح بالأصل الصحيح هنا في الخادم. ولاحظ: `http://localhost:3001` و `http://127.0.0.1:3001` أصلان مختلفان في نظر المتصفّح.

خطوة ٩ غيّر منفذ ال API إلى 3000 وثبّت الواجهة على 3001

قرار صغير يمنع تصادمًا يوميًا.

`Next.js` يستخدم `3000` افتراضيًا أيضًا. سنُبقى ال `API` على `3000` ونشغل الواجهة على `3001` لاحقًا بـ `next dev -p 3001`. المهم أن تختار وتلتزم، لأنّ العنوان يظهر في `CORS` وفي متغيّرات البيئة في أربعة أماكن.

خطوة ١٠ متغيّرات البيئة بشكل صحيح

`ConfigModule` مع تحقّق يمنع الإقلاع بإعداد ناقص.

```
npm i @nestjs/config joi
```

`api/.env`

```
NODE_ENV=development
PORT=3000
DATABASE_URL="postgresql://shop:shop_dev_pass@localhost:5433/minishop?schema=public"
JWT_ACCESS_SECRET=change_me_access_0123456789
JWT_REFRESH_SECRET=change_me_refresh_9876543210
JWT_ACCESS_TTL=15m
JWT_REFRESH_TTL=7d
WEB_ORIGIN=http://localhost:3001
```

وانسخه إلى ملف قالب يدخل `git` بقيم وهمية، ليعرف أي زميل (أو أنت بعد شهر) ما المطلوب:

```
cp .env .env.example
```

(ثم افتح `.env.example`. وامسح القيم الحقيقية، اترك المفاتيح فقط.)

الآن سجّل الوحدة مع تحقّق صارم:

```
import { Module } from "@nestjs/common";
import { ConfigModule } from "@nestjs/config";
import * as Joi from "joi";

@Module({
  imports: [
    ConfigModule.forRoot({
      isGlobal: true,
      validationSchema: Joi.object({
        NODE_ENV: Joi.string()
          .valid("development", "production", "test")
          .default("development"),
        PORT: Joi.number().default(3000),
        DATABASE_URL: Joi.string().required(),
        JWT_ACCESS_SECRET: Joi.string().min(16).required(),
        JWT_REFRESH_SECRET: Joi.string().min(16).required(),
        JWT_ACCESS_TTL: Joi.string().default("15m"),
        JWT_REFRESH_TTL: Joi.string().default("7d"),
        WEB_ORIGIN: Joi.string().uri().default("http://localhost:3001"),
      }),
    ),
  ],
})
export class AppModule {}
```

لماذا التحقق من البيئة يستحق العناية: بدونها، نسيان `JWT_ACCESS_SECRET` على خادم الإنتاج يعني أن التطبيق يقلع بنجاح ثم يفشل عند أول تسجيل دخول برسالة غامضة. معه، التطبيق **يرفض الإقلاع** ويقول لك أي مفتاح ناقص. الفشل المبكر الصريح أرخص دائمًا من الفشل المتأخر الغامض.

جربها الآن

احذف سطر `DATABASE_URL` مؤقتًا من `.env`. واحفظ. المتوقع أن ينهار التطبيق فورًا بـ:

```
Error: Config validation error: "DATABASE_URL" is required
```

أعد السطر واحفظ — يعود للعمل. الآن أنت تثق بإعدادك.

خطوة ١١ استخدم الإعدادات في main.ts بدل process.env

مصدر واحد للحقيقة، وقابل للاختبار.

api/src/main.ts

```
import { ConfigService } from "@nestjs/config";
app داخل bootstrap بعد إنشاء ...

const config = app.get(ConfigService);

app.enableCors({
  origin: [config.get<string>("WEB_ORIGIN")!],
  credentials: true,
});

const port = config.get<number>("PORT")!;
await app.listen(port);
```

`app.get(...)` يسحب مزودًا من حاوية الحقن يدويًا — مسموح هنا لأننا خارج أي صنف. داخل الخدمات والمتحكمات لا تفعل هذا أبدًا! احقن `ConfigService` في المُنشئ.

خطوة ١٢ جهّز ملف طلبات للتجارب

بديل خفيف عن Postman يعيش داخل المستودع.

api/requests.http

```
@host = http://localhost:3000/api

### فحص الحياة
GET {{host}}
```

مع إضافة `REST Client` ستظهر كلمة `Send Request` فوق كل طلب. سنضيف إلى هذا الملف طلبًا بعد كل ميزة، فيصير في النهاية توثيقًا حيًا للـ `API` كله ويدخل `git` مع الشيفرة — وهذا وحده يترك انطباعًا ممتازًا في مراجعة الكود.

تمرين المرحلة ٢

- أضف مسارًا `GET /api/health` في `AppController` يُرجع `{ status: "ok", time: new Date().toISOString() }`. جرّبه من المتصفح ومن `requests.http`.
- غيّر `setGlobalPrefix("api")` إلى `"v1"` ثم أعدها. لاحظ كم مكانًا يجب أن يتغيّر — هذا سبب وجود المتغيّر `@host` في ملف الطلبات.
- أضف متغيّر بيئة جديد `APP_NAME` مع تحقّق `required()`، واستخدمه في رسالة `console.log` عند الإقلاع.
- جرّب حذف `enableCors` بالكامل واحفظ. لن تلاحظ شيئًا الآن — ولن تلاحظ حتى المرحلة ٨. تذكر هذا الشعور.

قاعدة البيانات والهجرات

هنا تجد إجابة سؤالك: أين ال Models وأين ال Migrations في هذا العالم.

خطوة ١٣ تثبيت Prisma وهيئة

أداة واحدة تقوم بدور Eloquent والهجرات معًا.

```
npm i @prisma/client
npm i -D prisma
npx prisma init --datasource-provider postgresql
```

الأمر الأخير أنشأ `prisma/schema.prisma` وأضاف `DATABASE_URL` إلى `.env`. (عندنا موجود أصلاً، احذف المكرر إن ظهر).

أين ذهب ال Model؟ في `Laravel` عندك ثلاثة أشياء منفصلة: ملف `Migration` يصف تغيير الجدول، وصنف `Model` فيه العلاقات والخصائص، وفئة `Factory` للبيانات الوهمية. في `Prisma` يوجد ملف واحد هو `schema.prisma`: تصف فيه الشكل النهائي للجدول، ثم `Prisma` يستنتج الهجرة اللازمة بمقارنة الوصف بالحالة الحالية، ويولّد لك عملياً مطبوعاً بالكامل (`typed client`) يقوم بدور `Eloquent`.

هذا المشروع	Laravel
<code>prisma/migrations/ + prisma/schema.prisma</code> المولّد	<code>database/migrations/*.php</code>
لا يوجد — تعدّل المخطّط مباشرة	<code>php artisan make:migration</code>
<code>npx prisma migrate dev</code> (تطوير) / <code>migrate deploy</code> (إنتاج)	<code>php artisan migrate</code>
<code>model Product</code> داخل نفس ملف المخطّط	<code>app/Models/Product.php</code>
<code>prisma.product.findMany({ include: { category: true } })</code>	<code>Product::with('category')</code>
<code>npx prisma db seed</code>	<code>php artisan db:seed</code>
<code>npx prisma studio</code> (واجهة رسومية في المتصفح)	<code>php artisan tinker</code>

خطوة ١٤ اكتب المخطّط كاملاً

مستخدمون، أصناف، منتجات، ترجمات، ورموز تحديث.

افتح الملف واستبدل محتواه بالكامل. اقرأ التعليقات وأنت تكتب — كل سطر هنا سنستعمله لاحقاً.

```

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

enum Role {
  USER
  ADMIN
}

enum Locale {
  ar
  en
}

model User {
  id          Int      @id @default(autoincrement())
  email       String   @unique
  name        String
  password    String   مَجْرَأة دائِمة — لا تُخزّن كنص صريح أبداً
  role        Role     @default(USER)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  refreshTokens RefreshToken[]
  products       Product[] من أنشأ المنتج
}

model RefreshToken {
  id          Int      @id @default(autoincrement())
  tokenHash   String   @unique نخزّن التجزئة لا الرمز نفسه
  userId      Int
  user        User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  expiresAt   DateTime
  revokedAt   DateTime?
  createdAt   DateTime @default(now())

  @@index([userId])
}

model Category {
  id          Int      @id @default(autoincrement())
  slug        String   @unique معرّف ثابت لا يتغيّر بتغيّر اللغة
  createdAt   DateTime @default(now())

  translations CategoryTranslation[]
  products     Product[]
}

model CategoryTranslation {

```



```

id          Int          @id @default(autoincrement())
categoryId  Int
category    Category @relation(fields: [categoryId], references: [id], onDelete: Cascade)
locale      Locale
name        String

@@unique([categoryId, locale])  ترجمة واحدة لكل لغة لكل صنف
}

model Product {
  id          Int          @id @default(autoincrement())
  slug        String       @unique
  price       Decimal      @db.Decimal(10, 2)  لا تستعمل Float للنقود أبدًا
  stock       Int          @default(0)
  imageUrl    String?
  published   Boolean      @default(false)
  categoryId  Int
  category    Category @relation(fields: [categoryId], references: [id])
  createdById Int?
  createdBy   User?       @relation(fields: [createdById], references: [id], onDelete: SetNull)
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt

  translations ProductTranslation[]

  @@index([categoryId])
  @@index([published, createdAt])  فهرس مركّب لاستعلام القائمة الشائع
}

model ProductTranslation {
  id          Int          @id @default(autoincrement())
  productId   Int
  product     Product @relation(fields: [productId], references: [id], onDelete: Cascade)
  locale      Locale
  name        String
  description  String

  @@unique([productId, locale])
  @@index([name])
}

```

ثلاث قرارات في هذا المخطط تستحق أن تشرحها في مقابلة

- **جدول ترجمات منفصل** بدل عمودين `name_en` و `name_ar`. لماذا؟ لأنّ إضافة لغة ثالثة مع الأعمدة تعني هجرة تُعدّل كل جدول؛ ومع جدول الترجمات تعني صفوفًا جديدة فقط. الثمن: كل استعلام يحتاج `join`. مقايضة واعية، وهذا ما يُسأل عنه.
- **Decimal للسعر لا Float**. الأعداد العشرية الثنائية لا تمثّل 0.1 بدقة، فتتراكم فروق القروش. نفس السبب الذي يجعلك تستعمل `decimal(10,2)` في Laravel.
- **onDelete صريح**. حذف مستخدم يحذف رموز تحديثه (`Cascade`) لكن لا يحذف منتجاته — يجعل `createdById` فارغًا (`SetNull`). ترك هذا للافتراضي يعني مفاجآت في الإنتاج.

خطوة ١٥ شغل أول هجرة

من الوصف إلى جداول حقيقية.

```
npx prisma migrate dev --name init
```

ثلاثة أشياء حدثت بهذا الأمر الواحد:

1. قارن Prisma مخططك بحالة القاعدة، وولّد ملف SQL في `prisma/migrations/<timestamp>_init/migration.sql`.
2. نفّذ ذلك الملف على قاعدة البيانات.
3. ولّد عميل Prisma المطبوع داخل `node_modules` — ولهذا يعرف محرّك فورًا أنّ `product.price` من نوع `Decimal`.

جربها الآن

افتح ملف الهجرة المولّد واقرأه. سترى `CREATE TABLE` و `CREATE TYPE` و `CREATE INDEX` بلغة SQL صريحة. ثم افتح واجهة الاستعراض:

```
npx prisma studio
```

سيفتح المتصفح على `localhost:5555` وترى جداولك السبعة فارغة. هذا هو `tinker` لكن بواجهة.

قاعدة الهجرات

`migrate dev` للتطوير فقط — قد يطلب منك إعادة بناء القاعدة إن اكتشف انحرافًا، ويحذف البيانات. في الإنتاج تستعمل `migrate deploy` حصريًا: ينفّذ الهجرات الموجودة ولا يولّد شيئًا ولا يحذف شيئًا. ولا تعدّل ملف هجرة مُطبّقة على الإنتاج أبدًا — أنشئ هجرة جديدة تصحّحها.

خطوة ١٦ اربط Prisma بـ Nest عبر خدمة

اتصال واحد يُدار مع دورة حياة التطبيق.

```
npx nest g module prisma
npx nest g service prisma --no-spec
```

api/src/prisma/prisma.service.ts

```
import { Injectable, OnModuleInit, OnModuleDestroy } from "@nestjs/common";
import { PrismaClient } from "@prisma/client";

@Injectable()
export class PrismaService
  extends PrismaClient
  implements OnModuleInit, OnModuleDestroy
{
  async onModuleInit() {
    await this.$connect();    افتح الاتصال عند إقلاع التطبيق
  }
  async onModuleDestroy() {
    await this.$disconnect();  أغلقه بنظافة عند الإيقاف
  }
}
```

api/src/prisma/prisma.module.ts

```
import { Global, Module } from "@nestjs/common";
import { PrismaService } from "../prisma.service";

@Global()    متاح لكل الوحدات بلا استيراد متكرر
@Module({
  providers: [PrismaService],
  exports: [PrismaService],    بدون exports لا يستطيع أحد حقنه
})
export class PrismaModule {}
```

وأضيف `PrismaModule` إلى مصفوفة `imports` في `app.module.ts`.

أشهر خطأ في هذه الخطوة

Nest can't resolve dependencies of the XService (?). Please make sure that the argument
PrismaService at index [0] is available in the XModule context.

معناها الحرفي: نسيت `exports`، أو نسيت استيراد الوحدة. الرسالة مزعجة أول مرة، لكنها تخبرك بالضبط بالصنف والوحدة والموضع — اقرأها ولا تخمّن.

خطوة ١٧ اكتب ملف البذور

مدير وأصناف ومنتجات بلغتين — لتشتغل على بيانات لا على فراغ.

```
npm i -D tsx
npm i bcrypt
npm i -D @types/bcrypt
```

```

import { PrismaClient, Role, Locale } from "@prisma/client";
import * as bcrypt from "bcrypt";

const prisma = new PrismaClient();

async function main() {
  1) مستخدم مدير — upsert حتى لا يتكرر عند إعادة التشغيل
  const passwordHash = await bcrypt.hash("Admin@12345", 10);
  await prisma.user.upsert({
    where: { email: "admin@minishop.test" },
    update: {},
    create: {
      email: "admin@minishop.test",
      name: "Shop Admin",
      password: passwordHash,
      role: Role.ADMIN,
    },
  });

  2) صنفان، كل واحد باسمين
  const categories = [
    { slug: "electronics", ar: "إلكترونيات", en: "Electronics" },
    { slug: "books", ar: "كتب", en: "Books" },
  ];

  for (const c of categories) {
    await prisma.category.upsert({
      where: { slug: c.slug },
      update: {},
      create: {
        slug: c.slug,
        translations: {
          create: [
            { locale: Locale.ar, name: c.ar },
            { locale: Locale.en, name: c.en },
          ],
        },
      },
    });
  }

  3) منتجات تجريبية داخل صنف الإلكترونيات
  const electronics = await prisma.category.findUniqueOrThrow({
    where: { slug: "electronics" },
  });

  const products = [
    { slug: "wireless-mouse", price: 24.9, ar: "فأرة لاسلكية", en: "Wireless Mouse" },
    { slug: "usb-c-hub", price: 39.5, ar: "موزّع USB-C", en: "USB-C Hub" },
    { slug: "mech-keyboard", price: 89.0, ar: "لوحة مفاتيح ميكانيكية", en: "Mechanical Keyboard" },
  ];

  for (const p of products) {
    await prisma.product.upsert({

```

```

    where: { slug: p.slug },
    update: {},
    create: {
      slug: p.slug,
      price: p.price,
      stock: 10,
      published: true,
      categoryId: electronics.id,
      translations: {
        create: [
          { locale: Locale.ar, name: p.ar, description: `وصف ${p.ar}` },
          { locale: Locale.en, name: p.en, description: `Description of ${p.en}` },
        ],
      },
    },
  });
}

console.log("Seed done.");
}

main()
  .catch((e) => {
    console.error(e);
    process.exit(1);
  })
  .finally(() => prisma.$disconnect());

```

وسجّل الأمر في `package.json` (أضف المفتاح `prisma` في الجذر، بجانب `scripts`):

`api/package.json`

```

"prisma": {
  "seed": "tsx prisma/seed.ts"
}

```

جربها الآن

```
npx prisma db seed
```

المتوقّع: `Seed done.` ثم افتح `npx prisma studio` وتحقّق: جدول `User` فيه صف واحد، `Category` صفّان، `Product` ثلاثة، و `ProductTranslation` ستّة (ثلاثة منتجات × لغتين).

شغّل الأمر مرّة ثانية — يجب ألاّ يتكرّر شيء. هذه فائدة `upsert`.

upsert: «حدّث إن وُجد وإلاّ أنشئ». يقابل `updateOrCreate` في `Floquet`. استعماله في البذور يجعل الملف قابلاً لإعادة التشغيل (`idempotent`) — صفة مطلوبة في أي شيء يعمل على الإنتاج.

خطوة ١٨ أوامر Prisma التي ستحتاجها يوميًا

احفظ هذا الجدول، سيوفّر عليك بحثًا متكرّرًا.

الأمر	متى
<code>npx prisma migrate dev --name x</code>	بعد كل تعديل على المخطط، في التطوير
<code>npx prisma migrate deploy</code>	على خادم الإنتاج، ضمن خطوة النشر
<code>npx prisma generate</code>	بعد <code>git pull</code> غير المخطط، أو لتحديث الأنواع فقط
<code>npx prisma studio</code>	لرؤية البيانات وتعديلها يدويًا
<code>npx prisma migrate reset</code>	امسح كل شيء وأعد الهجرات والبذور (تطوير فقط)
<code>npx prisma format</code>	تنسيق ملف المخطط
<code>npx prisma db pull</code>	توليد مخطط من قاعدة بيانات موجودة مسبقًا

خطوة ١٩ اكتب هجرة ثانية بيدك

لتفهم الدورة كاملة، لا الأمر الأول فقط.

أضف حقلًا جديدًا إلى `Product`:

`sku String? @unique` رمز المنتج في المخزون

```
npx prisma migrate dev --name add_product_sku
```

جربها الآن

افتح مجلّد `prisma/migrations/`: صار فيه مجلّدان مرتّبان زمنياً. افتح الجديد — ستجد سطرين فقط: `ALTER TABLE ...` و `CREATE UNIQUE INDEX` و `ADD COLUMN "sku" TEXT;`. هذا هو تاريخ قاعدة بياناتك، ويدخل `git` مع الشيفرة تمامًا كما تفعل بهجرات `Laravel`.

لو أضفت حقلًا مطلوبًا على جدول فيه بيانات

سيسألك `Prisma` عن قيمة افتراضية، ولو لم تعطه سيقتراح إعادة بناء القاعدة (أي حذف البيانات). الحل الصحيح في مشروع حقيقي: أضف الحقل اختياريًا أولًا، املاه بهجرة بيانات، ثم اجعله مطلوبًا في هجرة ثالثة. ثلاث خطوات، بلا فقدان بيانات. هذا سؤال مقابلة متكرّر.

تمرين المرحلة ٣

1. أضف نموذجًا `Tag` بعلاقة كثير-كثير مع `Product`. ولّد الهجرة واقرأ ال `SQL`: كم جدولًا أنشأ؟ لماذا؟
2. أضف إلى البذور مستخدمًا عاديًا (`USER`) ببريد `user@minishop.test` وكلمة سر `User@12345`. ستحتاجه في المرحلة ٦.
3. شغّل `npx prisma migrate reset` ولاحظ أنّه يعيد تشغيل البذور تلقائيًا في النهاية.
4. في `Prisma Studio`، غيّر `published` لأحد المنتجات إلى `false` يدويًا. سنستعمل هذا في اختبار الفلترة لاحقًا.

أول مورد REST كامل

الأصناف: إنشاء وقراءة وتعديل وحذف، بتحقّق صحيح وأخطاء محترمة ورموز حالة صادقة.

خطوة ٢٠ وُلد المورد

أمر واحد ينشئ الوحدة والمتحكّم والخدمة وال DTOs.

```
npx nest g resource categories
```

اختر REST API، وأجب بـ **yes** على سؤال نقاط النهاية (CRUD entry points).

```
src/categories/
├─ dto/
│  ├─ create-category.dto.ts
│  └─ update-category.dto.ts
├─ entities/category.entity.ts  احذفه — Prisma يولّد أنواعنا
├─ categories.controller.ts
├─ categories.service.ts
├─ categories.module.ts
└─ categories.controller.spec.ts
```

لاحظ أنّ المولّد أضاف `CategoriesModule` تلقائيًا إلى `app.module.ts`. هذا يقابل تسجيل مسارات مورد كامل بسطر `Route::apiResource` في `Laravel`، لكن هنا تحصل على الملفات نفسها لا على المسارات فقط. احذف مجلّد `entities/` بالكامل: أنواعنا تأتي من `@prisma/client`، ولا داعي لتعريفها مزّين.

خطوة ٢١ اكتب ال DTO مع التحقّق

هذا هو ال `FormRequest` في عالم `Nest`.

```
npm i class-validator class-transformer
```


api/src/categories/dto/create-category.dto.ts

```
import { Type } from "class-transformer";
import {
  ArrayMinSize, IsArray, IsEnum, IsString, Matches,
  MaxLength, MinLength, ValidateNested,
} from "class-validator";
import { Locale } from "@prisma/client";

class TranslationDto {
  @IsEnum(Locale) // يقبل "en" أو "ar" فقط
  locale!: Locale;

  @IsString()
  @MinLength(2)
  @MaxLength(80)
  name!: string;
}

export class CreateCategoryDto {
  @IsString()
  @Matches(/^[a-z0-9-]+$/, {
    message: "slug must contain lowercase letters, digits and dashes only",
  })
  @MinLength(2)
  @MaxLength(60)
  slug!: string;

  @IsArray()
  @ArrayMinSize(1)
  @ValidateNested({ each: true }) // تحقق من كل عنصر داخل المصفوفة
  @Type(() => TranslationDto) // ضروري — بدون يبقى كائنًا عاديًا ولا يُتحقق منه
  translations!: TranslationDto[];
}
```

api/src/categories/dto/update-category.dto.ts

```
import { PartialType } from "@nestjs/mapped-types";
import { CreateCategoryDto } from "../create-category.dto";

// يجعل كل الحقول اختيارية مع الحفاظ على قواعد التحقق
export class UpdateCategoryDto extends PartialType(CreateCategoryDto) {}
```

الخطأ الأول لكل قادم من Laravel

نسيان `@Type(() => TranslationDto)`. النتيجة أنّ الكائنات المتداخلة تمرّ بلا تحقق إطلاقًا وأنت تظنّ أنّك محميّ. السبب أنّ TypeScript يمحو الأنواع عند التصريف، فلا يعرف `class-validator` نوع عناصر المصفوفة إلّا إذا أخبرته صراحة.

مقارنة مباشرة: `FormRequest::rules()` يعطيك مصفوفة قواعد نصيّة تُنفَّذ وقت الطلب. الـ `DTO` يعطيك **صنفاً حقيقيًا**: تحصل على التحقق و على النوع في المحرّر و على توثيق `Swagger` تلقائي لاحقًا، من نفس التعريف الواحد.

api/src/categories/categories.service.ts

```
import { Injectable, NotFoundException, ConflictException } from "@nestjs/common";
import { Prisma } from "@prisma/client";
import { PrismaService } from "../prisma/prisma.service";
import { CreateCategoryDto } from "../dto/create-category.dto";
import { UpdateCategoryDto } from "../dto/update-category.dto";

@Injectable()
export class CategoriesService {
  constructor(private readonly prisma: PrismaService) {}

  async create(dto: CreateCategoryDto) {
    try {
      return await this.prisma.category.create({
        data: {
          slug: dto.slug,
          translations: { create: dto.translations },
        },
        include: { translations: true },
      });
    } catch (e) {
      if (
        e instanceof Prisma.PrismaClientKnownRequestError &&
        e.code === "P2002"
      ) {
        // انتهاك قيد فريد
        throw new ConflictException(`Category slug "${dto.slug}" already exists`);
      }
      // أي خطأ آخر يصعد كما هو
      throw e;
    }
  }

  findAll() {
    return this.prisma.category.findMany({
      include: { translations: true, _count: { select: { products: true } } },
      orderBy: { id: "asc" },
    });
  }

  async findOne(id: number) {
    const category = await this.prisma.category.findUnique({
      where: { id },
      include: { translations: true },
    });
    if (!category) throw new NotFoundException(`Category ${id} not found`);
    return category;
  }

  async update(id: number, dto: UpdateCategoryDto) {
    await this.findOne(id);
    // يرمي 404 إن لم يوجد
  }
}
```

```

return this.prisma.$transaction(async (tx) => {
  if (dto.translations) {
    استبدال كامل للترجمات: احذف ثم أنشئ — أبسط وأصدق من الدمج الجزئي
    await tx.categoryTranslation.deleteMany({ where: { categoryId: id } });
    await tx.categoryTranslation.createMany({
      data: dto.translations.map((t) => ({ ...t, categoryId: id })),
    });
  }
  return tx.category.update({
    where: { id },
    data: { slug: dto.slug },
    include: { translations: true },
  });
});
}

async remove(id: number) {
  await this.findOne(id);
  const productCount = await this.prisma.product.count({
    where: { categoryId: id },
  });
  if (productCount > 0) {
    throw new ConflictException(
      `Cannot delete a category that still has ${productCount} product(s)`,
    );
  }
  await this.prisma.category.delete({ where: { id } });
}
}

```

المعاملة (\$transaction): الحذف ثم الإنشاء يجب أن ينجح معًا أو يفشل معًا؛ لو فشل الإنشاء بعد نجاح الحذف ليبقي الصنف بلا اسم. هذا هو نفس `DB::transaction()` عندك تمامًا. القاعدة واحدة في كل لغة: **كل كتابة متعددة الجداول تحتاج معاملة.**

api/src/categories/categories.controller.ts

```
import {
  Body, Controller, Delete, Get, HttpStatusCode, Param,
  ParseIntPipe, Patch, Post,
} from "@nestjs/common";
import { CategoriesService } from "../categories.service";
import { CreateCategoryDto } from "../dto/create-category.dto";
import { UpdateCategoryDto } from "../dto/update-category.dto";

@Controller("categories")      المسار الكامل: api/categories/
export class CategoriesController {
  constructor(private readonly categories: CategoriesService) {}

  @Post()                      201 تلقائيًا في Nest
  create(@Body() dto: CreateCategoryDto) {
    return this.categories.create(dto);
  }

  @Get()
  findAll() {
    return this.categories.findAll();
  }

  @Get(":id")
  findOne(@Param("id", ParseIntPipe) id: number) {
    return this.categories.findOne(id);
  }

  @Patch(":id")
  update(
    @Param("id", ParseIntPipe) id: number,
    @Body() dto: UpdateCategoryDto,
  ) {
    return this.categories.update(id, dto);
  }

  @Delete(":id")
  @HttpStatusCode(204)          حذف ناجح بلا محتوى
  remove(@Param("id", ParseIntPipe) id: number) {
    return this.categories.remove(id);
  }
}
```

الرمز عند النجاح	المسار	الفعل
201 Created	/api/categories	POST
200 OK	/api/categories	GET
200 أو 404	/api/categories/:id	GET
200	/api/categories/:id	PATCH
204 No Content	/api/categories/:id	DELETE

PUT أم PATCH؟

PUT يعني «استبدال المورد بالكامل بما أرسلته» — كل حقل ناقص يُمسح. **PATCH** يعني «عدّل ما أرسلته فقط». نحن نستعمل **PATCH** لأنّ الـ **DTO** عندنا جزئي. سؤال مقابلة شائع، وأغلب الناس يجيبون خطأ.

و**ParseIntPipe** يستحق وقفة: المعاملات في المسار نصوص دائماً. بدونهِ سيصل "5" إلى **Prisma** فيرمي خطأ نوع غامضاً. معه، يصل 5 رقمًا، و **/categories/abc** يُرفض بـ 400 برسالة واضحة قبل أن يلمس قاعدة البيانات.

api/requests.http

@host = http://localhost:3000/api

كل الأصناف

GET {{host}}/categories

صنف واحد

GET {{host}}/categories/1

إنشاء صنف

POST {{host}}/categories

Content-Type: application/json

```
{
  "slug": "clothing",
  "translations": [
    { "locale": "ar", "name": "ملابس" },
    { "locale": "en", "name": "Clothing" }
  ]
}
```

تعديل

PATCH {{host}}/categories/3

Content-Type: application/json

```
{ "translations": [
  { "locale": "ar", "name": "أزياء" },
  { "locale": "en", "name": "Fashion" } ] }
```

حذف

DELETE {{host}}/categories/3

أو بـ curl إن فضّلت الطرفيّة:

curl http://localhost:3000/api/categories

```
curl -X POST http://localhost:3000/api/categories \
-H "Content-Type: application/json" \
-d '{"slug":"clothing","translations":[{"locale":"ar","name":"ملابس"},
{"locale":"en","name":"Clothing"}]}'
```

جرب حالات الفشل أيضًا — وهذا أهم من حالة النجاح

ما ترسله	ما يجب أن يعود
GET /categories/999	404 رسالة Category 999 not found
GET /categories/abc	400 ParseIntPipe رفضه
slug باسم موجود	409 Conflict — التقطنا P2002
slug: "Has Spaces"	400 رسالة قاعدة @Matches
حقل زائد "color": "red"	400 forbidNonWhitelisted
translations: []	400 ArrayMinSize(1)
حذف صنف فيه منتجات	409 رسالة صريحة

شكل خطأ التحقق الذي يعيده Nest:

```
{
  "message": [
    "slug must contain lowercase letters, digits and dashes only",
    "translations.0.locale must be one of the following values: ar, en"
  ],
  "error": "Bad Request",
  "statusCode": 400
}
```

خطوة ٢٥ فلتير أخطاء عام

حتى لا تتسرّب تفاصيل داخلية إلى العميل.

التقاط P2002 في كل خدمة سيتكرّر مملًا. الأفضل فلتير واحد يترجم أخطاء Prisma المعروفة إلى أخطاء HTTP مفهومة، ويحوّل أي شيء غير متوقّع إلى 500 بلا كشف تفاصيل.

```
npx nest g filter common/filters/prisma-exception --flat --no-spec
```

api/src/common/filters/prisma-exception.filter.ts

```
import {
  ArgumentsHost, Catch, ExceptionFilter, HttpStatus, Logger,
} from "@nestjs/common";
import { Prisma } from "@prisma/client";
import { Response } from "express";

@Catch(Prisma.PrismaClientKnownRequestError)
export class PrismaExceptionFilter implements ExceptionFilter {
  private readonly logger = new Logger(PrismaExceptionFilter.name);

  catch(e: Prisma.PrismaClientKnownRequestError, host: ArgumentsHost) {
    const res = host.switchToHttp().getResponse<Response>();

    switch (e.code) {
      case "P2002": قيد فريد
        return res.status(HttpStatus.CONFLICT).json({
          statusCode: 409,
          error: "Conflict",
          message: `Duplicate value for: ${e.meta?.target as string[]?.join(", ")}`,
        });

      case "P2025": السجل المطلوب غير موجود
        return res.status(HttpStatus.NOT_FOUND).json({
          statusCode: 404, error: "Not Found", message: "Record not found",
        });

      case "P2003": انتهاك مفتاح أجنبي
        return res.status(HttpStatus.BAD_REQUEST).json({
          statusCode: 400, error: "Bad Request",
          message: "Related record does not exist",
        });

      default:
        this.logger.error(`Unhandled Prisma error ${e.code}`, e.stack);
        return res.status(HttpStatus.INTERNAL_SERVER_ERROR).json({
          statusCode: 500, error: "Internal Server Error",
          message: "Something went wrong", لا تكشف تفاصيل القاعدة للعميل
        });
    }
  }
}
```

سجله عالميًا في `main.ts`:

```
app.useGlobalFilters(new PrismaExceptionFilter());
```

قاعدة أمنية عامة

رسالة الخطأ للعميل تقول ماذا فشل، والسجل الداخلي يقول لماذا. اسم جدول أو استعلام `SQL` يظهر في استجابة عامة هو تسريب معلومات، وهو من أوّل ما يبحث عنه المُدقّق الأمني.

خطوة ٢٦ قرار: هل نغلف الاستجابة؟

سؤال يستحق موقفًا واضحًا لأنه يُسأل في المقابلات.

بعض الفرق تغلف كل استجابة: { "data": ..., "meta": ... }. وبعضها يعيد الكائن مباشرة.

غلف بـ data	أعيد الكائن مباشرة
مكان جاهز للـ meta (الترقيم، التحذيرات)	أبسط للاستهلاك، أقل ضجيج
شكل ثابت يسهل معالج الاتصال في الواجهة	يوافق أغلب أمثلة الوثائق
ثمنه سطر إضافي في كل استهلاك	تغييره لاحقًا كسر تغييراتي

قرارنا في هذا المشروع: الكائن مباشرة للمورد الواحد، وغلاف { items, meta } للقوائم المرقمة فقط. المهم ليس أي خيار تختار، بل أن تكون متسقًا وأن تعرف لماذا اخترت.

خطوة ٢٧ نظف: احذف ما لا تستعمله

مستودع نظيف انطباع أول ممتاز.

- احذف src/app.controller.ts و src/app.service.ts وملف الاختبار المرافق، وأزل ذكرهما من app.module.ts — إلا إن أبقى مسار health / من تمرين المرحلة ٢.
- احذف مجلدات entities/ المولدة.
- git add . && git commit -m "feat(categories): full CRUD resource"

تمرين المرحلة ٤

- أضف نقطة نهاية GET /api/categories/slug/:slug تُرجع الصنف بالمُعَرَّف النصّي بدل الرقم. (انتبه لترتيب المسارات: ضعها قبل @Get(":id") وفكّر لماذا.)
- أضف قاعدة تحقّق تمنع تكرار نفس اللغة مرّتين في مصفوفة translations. (تلميح: مُحَقِّق مخصّص، أو فحص داخل الخدمة قبل الإنشاء — أيّهما أنسب ولماذا؟)
- غيّر @HttpCode(204) إلى 200 واجعل remove يُرجع الكائن المحذوف. أيّهما أفضل برأيك؟ ابحث عن حجة للطرفين.
- أرسل طلب إنشاء بـ Content-Type: text/plain. ماذا يحدث؟ ولماذا؟

REST بمعناه الكامل

قائمة منتجات حقيقية: ترقيم وبحث وفلتر وترتيب وعلاقات ورفع صور.

خطوة ٢٨ وُلد مورد المنتجات

نفس الأمر، لكن المحتوى أعمق هذه المرة.

```
npx nest g resource products
```

```
api/src/products/dto/create-product.dto.ts
```

```
import { Type } from "class-transformer";
import {
  ArrayMinSize, IsArray, IsBoolean, IsEnum, IsInt, IsNumber,
  IsOptional, IsString, Matches, Max, MaxLength, Min, MinLength,
  ValidateNested,
} from "class-validator";
import { Locale } from "@prisma/client";

class ProductTranslationDto {
  @IsEnum(Locale) locale!: Locale;
  @IsString() @MinLength(2) @MaxLength(120) name!: string;
  @IsString() @MinLength(10) @MaxLength(2000) description!: string;
}

export class CreateProductDto {
  @IsString() @Matches(/^[-a-z0-9-]+$/) @MinLength(2) @MaxLength(80)
  slug!: string;

  @IsNumber({ maxDecimalPlaces: 2 })
  @Min(0) @Max(1_000_000)
  @Type(() => Number)
  price!: number;

  @IsInt() @Min(0) @IsOptional() @Type(() => Number)
  stock?: number;

  @IsInt() @Min(1) @Type(() => Number)
  categoryId!: number;

  @IsBoolean() @IsOptional()
  published?: boolean;

  @IsArray() @ArrayMinSize(1)
  @ValidateNested({ each: true })
  @Type(() => ProductTranslationDto)
  translations!: ProductTranslationDto[];
}
```

معاملات الاستعلام تأتي دائمًا نصوصًا: `?page=2` يصل كـ "2". `@Type(() => Number)` مع `transform: true` في الـ `ValidationPipe` يحولها قبل التحقق. هذه من أكثر النقاط التي تُربك المبتدئين.

api/src/products/dto/query-products.dto.ts

```
import { Transform, Type } from "class-transformer";
import { IsBoolean, IsEnum, IsInt, IsOptional, IsString, Max, Min } from "class-validator";

export enum ProductSort {
  newest = "newest",
  priceAsc = "price_asc",
  priceDesc = "price_desc",
  nameAsc = "name_asc",
}

export class QueryProductsDto {
  @IsOptional() @Type(() => Number) @IsInt() @Min(1)
  page: number = 1;

  @IsOptional() @Type(() => Number) @IsInt() @Min(1) @Max(100)
  limit: number = 12;          سقف صارم — بلا سقف يستطيع أحدهم طلب مليون صف

  @IsOptional() @IsString()
  search?: string;             بحث في اسم المنتج

  @IsOptional() @Type(() => Number) @IsInt()
  categoryId?: number;

  @IsOptional() @Type(() => Number) @Min(0)
  minPrice?: number;

  @IsOptional() @Type(() => Number) @Min(0)
  maxPrice?: number;

  "true"/"false" لـ Boolean("false") = true
  @IsOptional()
  @Transform(({ value }) => value === "true" || value === true)
  @IsBoolean()
  published?: boolean;

  @IsOptional() @IsEnum(ProductSort)
  sort: ProductSort = ProductSort.newest;
}
```

فجّ `Boolean("false")`

في JavaScript أي نص غير فارغ قيمته صادقة، فـ `Boolean("false")` تساوي `true`. لو اعتمدت على التحويل التلقائي لصار `published=false` يعني «المنشورة». لهذا كتبنا `@Transform` يدويًا. هذا سؤال مقابلة كلاسيكي عن الـ `truthy/falsy`.

api/src/products/products.service.ts

```
import { Injectable, NotFoundException } from "@nestjs/common";
import { Prisma } from "@prisma/client";
import { PrismaService } from "../../prisma/prisma.service";
import { ProductSort, QueryProductsDto } from "../../dto/query-products.dto";

@Injectable()
export class ProductsService {
  constructor(private readonly prisma: PrismaService) {}

  async findAll(q: QueryProductsDto) {
    (1) ابن شرط WHERE قطعة قطعة — أضف فقط ما أرسل فعلاً
    const where: Prisma.ProductWhereInput = {};

    if (q.published !== undefined) where.published = q.published;
    if (q.categoryId) where.categoryId = q.categoryId;

    if (q.minPrice !== undefined || q.maxPrice !== undefined) {
      where.price = {
        ...(q.minPrice !== undefined ? { gte: q.minPrice } : {}),
        ...(q.maxPrice !== undefined ? { lte: q.maxPrice } : {}),
      };
    }

    if (q.search) {
      where.translations = {
        some: {
          (يوجد ترجمة واحدة على الأقل تطابق)
          name: { contains: q.search, mode: "insensitive" },
        },
      };
    }

    (2) الترتيب
    const orderBy: Prisma.ProductOrderByWithRelationInput =
      q.sort === ProductSort.priceAsc ? { price: "asc" } :
      q.sort === ProductSort.priceDesc ? { price: "desc" } :
      q.sort === ProductSort.nameAsc ? { slug: "asc" } :
      { createdAt: "desc" };

    (3) العدّ والصفوف في رحلة واحدة إلى القاعدة
    const skip = (q.page - 1) * q.limit;
    const [total, items] = await this.prisma.$transaction([
      this.prisma.product.count({ where }),
      this.prisma.product.findMany({
        where,
        orderBy,
        skip,
        take: q.limit,
        include: {
          translations: true,

```

```

        category: { include: { translations: true } } },
    },
  )),
]);

return {
  items,
  meta: {
    total,
    page: q.page,
    limit: q.limit,
    pages: Math.ceil(total / q.limit),
    hasNext: skip + items.length < total,
  },
};
}

async findOne(id: number) {
  const product = await this.prisma.product.findUnique({
    where: { id },
    include: { translations: true, category: { include: { translations: true } } },
  });
  if (!product) throw new NotFoundException(`Product ${id} not found`);
  return product;
}
}

```

لماذا \$transaction هنا وليس استعلامين منفصلين؟ لأنّ العدّ والصفوف يجب أن يريا نفس اللقطة من البيانات. لو أدرج منتج بين الاستعلامين لصار **total** غير متّسق مع الصفحة المعروضة. الفائدة الثانية: رحلة شبكة واحدة إلى القاعدة بدل اثنتين.

مقارنة بـ Eloquent

Product::where(...)->paginate(12) يفعل هذا كلّ من أجلك ويعيد كائن ترقيم جاهزًا. في **Prisma** لا يوجد ترقيم مدمج — تكتبه بنفسك بـ **skip/take** و **count**. أطول قليلًا، لكنك ترى ما يحدث بالضبط. اكتبه مرّة في دالة مساعدة وأعد استعماله.

api/src/products/products.controller.ts

```
@Controller("products")
export class ProductsController {
  constructor(private readonly products: ProductService) {}

  @Get()
  findAll(@Query() query: QueryProductsDto) {
    return this.products.findAll(query);
  }

  @Get(":id")
  findOne(@Param("id", ParseIntPipe) id: number) {
    return this.products.findOne(id);
  }
}
```

جربها الآن

```
curl "http://localhost:3000/api/products"
curl "http://localhost:3000/api/products?page=1&limit=2"
curl "http://localhost:3000/api/products?search=mouse"
curl "http://localhost:3000/api/products?search=فأرة"
curl "http://localhost:3000/api/products?minPrice=30&maxPrice=50"
curl "http://localhost:3000/api/products?sort=price_desc"
curl "http://localhost:3000/api/products?published=true&categoryId=1"
```

لاحظ أنّ البحث العربي يعمل أيضًا — لأننا نبحث في جدول الترجمات لا في عمود واحد.
شكل الاستجابة المتوقع:

```
{
  "items": [ { "id": 3, "slug": "mech-keyboard", "price": "89", ... } ],
  "meta": { "total": 3, "page": 1, "limit": 2, "pages": 2, "hasNext": true }
}
```

ثم جرب الرفض: `limit=5000` يجب أن يعطي 400 بسبب `@Max(100)`، و `sort=random` يعطي 400 بسبب `@IsEnum`.

لماذا السعر يعود كنص "89" لا رقمًا؟

لأنّ نوع `Decimal` لا يُمثّل بدقّة في عدد `JavaScript`، فيعيده `Prisma` كائنًا خاصًا يتحوّل إلى نص عند التسلسل إلى `JSON`. هذا صحيح ومقصود. في الواجهة حوّله بـ `Number(p.price)` عند العرض فقط، ولا تُجر عليه حسابات مالية في المتصفح.

خطوة ٣٢ افهم N+1 قبل أن تقع فيه

نفس المشكلة التي تعرفها من Eloquent، بوجه جديد.

الكود التالي يبدو بريئاً وهو كارثة:

```
const products = await prisma.product.findMany({ take: 50 });
```

خطأ: استعلام لكل منتج — 51 رحلة إلى قاعدة البيانات

```
for (const p of products) {  
  p.category = await prisma.category.findUnique({ where: { id: p.categoryId } });  
}
```

الصواب استعلام واحد:

```
const products = await prisma.product.findMany({  
  take: 50,  
  include: { category: true }, ← Eloquent في with('category')  
});
```

وإن أردت أعمدة محدّدة فقط (أخفّ على الشبكة وعلى القاعدة):

```
const products = await prisma.product.findMany({  
  select: {  
    id: true, slug: true, price: true,  
    category: { select: { slug: true } },  
    translations: { select: { locale: true, name: true } },  
  },  
});
```

القاعدة

`include` = «أحضِر العلاقة كاملة مع كل حقول المورد الأصلي». `select` = «أحضِر هذه الحقول فقط». لا يمكن استعمالهما معاً على نفس المستوى. وعلامة الخطر التي تدلّك على `N+1`: `await` داخل حلقة. كلّما رأيتهَا، توقّف وأعد التفكير.

شاهد الاستعلامات بعينك

فعل تسجيل الاستعلامات مؤقتاً في `PrismaService`:

```
super({ log: ["query"] }); // داخل constructor
```

أعد تشغيل الخادم واطلب قائمة المنتجات. ستطبع الطرفيّة كل استعلام `SQL` فعلي. غُدّها: يجب أن تكون قليلة وثابتة مهما زاد عدد المنتجات. أطفئ التسجيل بعد التجربة — إنّه ثرثار جدّاً.

```
npm i -D @types/multer
```

api/src/products/products.controller.ts

```
import { FileInterceptor } from "@nestjs/platform-express";
import { diskStorage } from "multer";
import { extname } from "path";
import { BadRequestException, UploadedFile, UseInterceptors } from "@nestjs/common";

@Post(":id/image")
@UseInterceptors(
  FileInterceptor("file", {
    storage: diskStorage({
      destination: "./uploads",
      filename: (_req, file, cb) => {
        const unique = `${Date.now()}-${Math.round(Math.random() * 1e9)}`;
        cb(null, `${unique}${extname(file.originalname)}`);
      },
    }),
    limits: { fileSize: 2 * 1024 * 1024 },    سقف ٢ ميغابايت
    fileFilter: (_req, file, cb) => {
      const ok = ["image/jpeg", "image/png", "image/webp"].includes(file.mimetype);
      cb(ok ? null : new BadRequestException("Only JPEG, PNG or WebP"), ok);
    },
  }),
)
uploadImage(
  @Param("id", ParseIntPipe) id: number,
  @UploadedFile() file: Express.Multer.File,
) {
  if (!file) throw new BadRequestException("File is required");
  return this.products.setImage(id, `/uploads/${file.filename}`);
}
```

وفي الخدمة:

```
async setImage(id: number, imageUrl: string) {
  await this.findOne(id);
  return this.prisma.product.update({ where: { id }, data: { imageUrl } });
}
```

وأخيرًا اجعل الملفات المرفوعة قابلة للوصول — في `main.ts`:


```
import { NestExpressApplication } from "@nestjs/platform-express";
import { join } from "path";

const app = await NestFactory.create<NestExpressApplication>(AppModule);

app.useStaticAssets(join(__dirname, "..", "uploads"), {
  prefix: "/uploads/",    لاحظ: خارج بادئة api/ عمدًا
});
```

جربها الآن

```
curl -X POST http://localhost:3000/api/products/1/image \
-F "file=@/path/to/photo.jpg"
```

المتوقع: استجابة فيها "imageUrl": "/uploads/1718...-123456789.jpg". افتح <http://localhost:3000/uploads/<الاسم>> في المتصفح — يجب أن ترى الصورة. ثم جرب رفض الملفات: ارفع ملف `txt`. (يجب 400) وملفًا أكبر من ٢ ميغابايت (يجب 413).

لا تخزن الملفات على القرص في الإنتاج

الخوادم الحديثة (Vercel، Railway، أي حاوية) نظام ملفات زائل — تُمسح عند كل نشر أو إعادة تشغيل. في الإنتاج ترفع إلى تخزين كائنات (S3، Cloudflare R2، Supabase Storage) وتخزن الرابط فقط. منطق المتحكم يبقى كما هو؛ يتغير الـ storage فقط. اذكر هذا في المقابلة، فهو فرق الخبرة.

خطوة ٣٤ أكمل CRUD المنتجات

تطبيق ما تعلّمته في المرحلة ٤ بلا مساعدة.

اكتب بنفسك `create` و `update` و `remove` للمنتجات في الخدمة والمتحكم، على غرار ما فعلناه في الأصناف: معاملة لاستبدال الترجمات، 404 عند عدم الوجود، 409 عند تكرار slug، و 204 عند الحذف. نقطة إضافية واحدة تخصّ المنتجات: تحقق أنّ `categoryId` موجود فعليًا قبل الإنشاء، وإلا سيرمي Prisma خطأ مفتاح أجنبي غامضًا:

```
const exists = await this.prisma.category.count({ where: { id: dto.categoryId } });
if (!exists) throw new BadRequestException(`Category ${dto.categoryId} does not exist`);
```

خطوة ٣٥ وثّق الطلبات واحفظ التقدّم

أضف إلى `requests.http` طلبًا لكل حالة جربتها: القائمة، الفلترة، البحث، الترتيب، الإنشاء، التعديل، الحذف، رفع الصورة.

```
git add . && git commit -m "feat(products): list with pagination, filters, search and image upload"
```

تمرين المرحلة ٥

1. أضف فلتر `inStock=true` يعيد المنتجات التي `stock > 0` فقط.
2. أضف خيار ترتيب `name_asc` يترتب فعليًا حسب الاسم في لغة الطلب لا حسب الـ `slug`. (صعب: يحتاج ترتيبًا عبر علاقة — ابحث عن `orderBy` على علاقة في وثائق Prisma).
3. استخرج منطق الترقيم إلى دالة مساعدة `paginate()` في `src/common/` واستعملها في الأصناف والمنتجات.
4. أضف نقطة `DELETE /api/products/:id/image` تسمح الصورة من القرص ومن قاعدة البيانات. ماذا يحدث لو حُذف الصف ونجح، وفشل حذف الملف؟ أيّهما تنفّذ أولًا ولماذا؟
5. افتح `?page=999`. ماذا يعيد؟ هل هذا سلوك جيّد أم يجب أن يكون `404`؟

المصادقة والصلاحيات

تسجيل ودخول برمز JWT، تجديد بالتدوير، حارس عام، وأدوار. أطول مرحلة وأهمّها.

الصورة الكاملة قبل الكود

نقطة النهاية	ما يحدث بالترتيب
POST /auth/register	تجزئة كلمة السر → إنشاء المستخدم → إصدار رمزين
POST /auth/login	البحث بالبريد → مقارنة التجزئة → إصدار رمزين
GET /products (محمي)	JwtAuthGuard يفكّ الرمز → يضع المستخدم في req.user → المتحكم
POST /auth/refresh	التحقق من التجزئة → إبطال الرمز القديم → إصدار زوج جديد
POST /auth/logout	إبطال رمز التجديد في قاعدة البيانات

الرمز	العمر	أين يعيش
access token	قصير — ١٥ دقيقة	يُرسل في ترويسة Authorization، ولا يُخزّن في الخادم
refresh token	طويل — ٧ أيام	يُخزّن مجزّأً في جدول RefreshToken فيمكن إبطاله

لماذا رمزان لا رمز واحد؟ رمز طويل العمر مريح لكن سرّته كارثة لا يمكن إيقافها لأنّ JWT لا يُلغى. الحل: رمز وصول قصير جدًّا (لو سُرق انتهى خلال دقائق) ورمز تجديد طويل لكنه مخزّن في القاعدة فيمكن إبطاله فورًا. هذا سؤال مقابلة شبه مؤكّد — احفظ الجواب بمنطقه لا بحرفه.

خطوة ٣٦ تثبيت الحزم وولّد الوحدة

```
npm i @nestjs/jwt @nestjs/passport passport passport-jwt
npm i -D @types/passport-jwt
npx nest g module auth
npx nest g service auth --no-spec
npx nest g controller auth --no-spec
```

خطوة ٣٧ تجزئة كلمة السر

القاعدة الوحيدة التي لا تُكسر أبدًا.

كلمة السر تُجزّأ ولا تُشفر. الفرق: التشفير عكوس، والتجزئة ليست كذلك. نستعمل bcrypt (تُبتناه في المرحلة ٣) بمعامل كلفة 10. bcrypt يولّد ملحًا (salt) فريدًا داخليًا ويخزّنه ضمن التجزئة نفسها — لهذا كلمتا سر متطابقتان تعطيان تجزئتين مختلفتين، وهذا مطلوب.

api/src/auth/auth.service.ts

```
import * as bcrypt from "bcrypt";
...
private hash(plain: string) {
  return bcrypt.hash(plain, 10);
}
private compare(plain: string, hashed: string) {
  return bcrypt.compare(plain, hashed);
}
```

ثلاثة أخطاء تُسقط المرشح فورًا

تخزين كلمة السر كنص صريح · استعمال md5 أو sha1 (سريعة جدًا فتُكسر بالقوّة الغاشمة) · إعادة حقل password ضمن أي استجابة JSON. الثالث هو الأكثر حدوثًا — سنعالجه صراحةً بعد قليل.

التسجيل خطوة ٣٨

api/src/auth/dto/register.dto.ts

```
import { IsEmail, IsString, Matches, MaxLength, MinLength } from "class-validator";

export class RegisterDto {
  @IsEmail() email!: string;

  @IsString() @MinLength(2) @MaxLength(60)
  name!: string;

  @IsString() @MinLength(8) @MaxLength(72)
  @Matches(/(?=.*[a-z])(?=.*[A-Z])(?=.*\d)/, {
    message: "password must contain lowercase, uppercase and a digit",
  })
  password!: string;
}
```

لماذا `MaxLength(72)`؟ لأنّ `bcrypt` يتجاهل ما بعد ٧٢ بايت بصمت — تفصيلاً يعرفها القليلون.

```
async register(dto: RegisterDto) {
  const exists = await this.prisma.user.count({ where: { email: dto.email } });
  if (exists) throw new ConflictException("Email already registered");

  const user = await this.prisma.user.create({
    data: { email: dto.email, name: dto.name, password: await this.hash(dto.password) },
  });
  return this.issueTokens(user.id, user.email, user.role);
}
```

```

async login(dto: LoginDto) {
  const user = await this.prisma.user.findUnique({ where: { email: dto.email } });

  نفس الرسالة في الحالتين — لا تكشف أي البريد مسجل
  if (!user || !(await this.compare(dto.password, user.password))) {
    throw new UnauthorizedException("Invalid credentials");
  }
  return this.issueTokens(user.id, user.email, user.role);
}

private async issueTokens(userId: number, email: string, role: Role) {
  const payload = { sub: userId, email, role };

  const accessToken = await this.jwt.signAsync(payload, {
    secret: this.config.get("JWT_ACCESS_SECRET"),
    expiresIn: this.config.get("JWT_ACCESS_TTL"),
  });

  const refreshToken = await this.jwt.signAsync(payload, {
    secret: this.config.get("JWT_REFRESH_SECRET"),
    expiresIn: this.config.get("JWT_REFRESH_TTL"),
  });

  خزن تجزئة رمز التجديد — لا الرمز نفسه
  await this.prisma.refreshToken.create({
    data: {
      tokenHash: await this.hash(refreshToken),
      userId,
      expiresAt: new Date(Date.now() + 7 * 24 * 60 * 60 * 1000),
    },
  });

  return { accessToken, refreshToken };
}

```

لماذا نفس الرسالة للبريد الخاطئ وللمرء السر الخاطئة؟

لو قلت «البريد غير مسجل» لأعطيت المهاجم أداة مجانية لمعرفة من هم مستخدموك (`user enumeration`). القاعدة: **افشل بنفس الطريقة دائماً**. وللسبب نفسه لا تجعل الرد أسرع في حالة عن أخرى.

وسجل `JwtModule` في وحدة المصادقة:

api/src/auth/auth.module.ts

```
@Module({
  imports: [
    PassportModule,
    JwtModule.register({}),    فارغ عمدًا — نمزّر السرّ عند كل توقيع
  ],
  controllers: [AuthController],
  providers: [AuthService, JwtStrategy],
})
export class AuthModule {}
```

خطوة ٤٠ استراتيجية JWT والحارس

من يقرأ الرمز ويضع المستخدم في الطلب.

api/src/auth/strategies/jwt.strategy.ts

```
import { Injectable, UnauthorizedException } from "@nestjs/common";
import { PassportStrategy } from "@nestjs/passport";
import { ExtractJwt, Strategy } from "passport-jwt";
import { ConfigService } from "@nestjs/config";
import { PrismaService } from "../../prisma/prisma.service";

export type JwtPayload = { sub: number; email: string; role: "USER" | "ADMIN" };

@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy, "jwt") {
  constructor(config: ConfigService, private readonly prisma: PrismaService) {
    super({
      jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
      ignoreExpiration: false,
      secretOrKey: config.get<string>("JWT_ACCESS_SECRET")!,
    });
  }

  req.user تُستدعى بعد التحقق من التوقيع والصلاحيّة — ما تُرجعه يصير
  async validate(payload: JwtPayload) {
    const user = await this.prisma.user.findUnique({
      where: { id: payload.sub },
      select: { id: true, email: true, name: true, role: true },
    });
    if (!user) throw new UnauthorizedException();
    return user;
  }
}
```

لماذا نستعلم عن المستخدم مع أنّ الرمز يحمل بياناته؟ لأنّ الرمز صورة مجمّدة لحظة الإصدار. لو حُذف المستخدم أو حُفّض دوره من ADMIN إلى USER، فرمزه القديم ما زال يقول ADMIN حتى ينتهي. الاستعلام يكلفك رحلة واحدة إلى القاعدة ويشترى لك صحّة فورية. مقايضة واعية — اذكرها في المقابلة، فهي تدلّ على فهم لا على حفظ.

api/src/auth/guards/jwt-auth.guard.ts

```
import { ExecutionContext, Injectable } from "@nestjs/common";
import { Reflector } from "@nestjs/core";
import { AuthGuard } from "@nestjs/passport";
import { IS_PUBLIC_KEY } from "../../decorators/public.decorator";

@Injectable()
export class JwtAuthGuard extends AuthGuard("jwt") {
  constructor(private reflector: Reflector) { super(); }

  canActivate(context: ExecutionContext) {
    const isPublic = this.reflector.getAllAndOverride<boolean>(IS_PUBLIC_KEY, [
      context.getHandler(),    المزخرف على الدالة
      context.getClass(),      أو على الصنف كُله
    ]);
    if (isPublic) return true;
    return super.canActivate(context);
  }
}
```

خطوة ٤١ اجعل الحماية هي الافتراض

حارس عام + مزخرف Public — أهم قرار أمني في المشروع.

api/src/auth/decorators/public.decorator.ts

```
import { SetMetadata } from "@nestjs/common";
export const IS_PUBLIC_KEY = "isPublic";
export const Public = () => SetMetadata(IS_PUBLIC_KEY, true);
```

وسجّل الحارس عالميًا في `app.module.ts`:

```
import { APP_GUARD } from "@nestjs/core";

providers: [
  { provide: APP_GUARD, useClass: JwtAuthGuard },
]
```

الآن كل نقطة نهاية محمية، وتفتح ما تريد فتحه صراحةً:

```
@Public()
@Post("login")
login(@Body() dto: LoginDto) { return this.auth.login(dto); }
```

لماذا هذا الاتجاه وليس العكس

لو كان الافتراض «مفتوح» ثم تحمي ما تريد، فنسيان مزخرف واحد يعني ثغرة صامتة لا يكتشفها أحد. ومع الافتراض «محمي»، نسيان `@Public()` يعني أنّ الصفحة تعطي 401 فورًا في أول تجربة — عطل ظاهر خير من ثغرة خفية. طبق هذا المبدأ في كل مشروع.

افتح المسارات العامة الآن: `refresh`, `login`, `register`، وقائمة المنتجات والأصناف للقراءة.

خطوة ٤٢ مزخرف للمستخدم الحالي

بديل نظيف عن `req.user` المتناثر.

`api/src/auth/decorators/current-user.decorator.ts`

```
import { createParamDecorator, ExecutionContext } from "@nestjs/common";

export type AuthUser = { id: number; email: string; name: string; role: "USER" | "ADMIN" };

export const CurrentUser = createParamDecorator(
  (data: keyof AuthUser | undefined, ctx: ExecutionContext) => {
    const user = ctx.switchToHttp().getRequest().user as AuthUser;
    return data ? user?.[data] : user;
  },
);
```

الاستعمال:

```
@Get("me")
me(@CurrentUser() user: AuthUser) { return user; }

@Post()
create(@Body() dto: CreateProductDto, @CurrentUser("id") userId: number) {
  return this.products.create(dto, userId);
}
```

هذا يقابل `auth()->user()` في `Laravel`، لكنه مُصرَّح النوع فيصَحِّحك المحرِّر عند الخطأ.

خطوة ٤٣ الأدوار

من يستطيع الكتابة، ومن يقرأ فقط.

`api/src/auth/decorators/roles.decorator.ts`

```
import { SetMetadata } from "@nestjs/common";
import { Role } from "@prisma/client";
export const ROLES_KEY = "roles";
export const Roles = (...roles: Role[]) => SetMetadata(ROLES_KEY, roles);
```



```

@Inject()
export class RolesGuard implements CanActivate {
  constructor(private reflector: Reflector) {}

  canActivate(ctx: ExecutionContext): boolean {
    const required = this.reflector.getAllAndOverride<Role[]>(ROLES_KEY, [
      ctx.getHandler(), ctx.getClass(),
    ]);
    if (!required?.length) return true;    لا أدوار مطلوبة — اسمح

    const { user } = ctx.switchToHttp().getRequest();
    if (!user) throw new UnauthorizedException();    401
    if (!required.includes(user.role)) {
      throw new ForbiddenException("Insufficient role");    403
    }
    return true;
  }
}

```

سجله عالميًا أيضًا (بعد حارس JWT — الترتيب مهم لأنه يحتاج req.user جاهزًا):

```

providers: [
  { provide: APP_GUARD, useClass: JwtAuthGuard },
  { provide: APP_GUARD, useClass: RolesGuard },
]

```

ثم احم كتابة المنتجات:

```

@Roles(Role.ADMIN)
@Post()
create(@Body() dto: CreateProductDto) { ... }

```

401 أم 403؟ Unauthorized 401 = «لا أعرف من أنت» — لا رمز، أو رمز منتهٍ أو فاسد. Forbidden 403 = «أعرفك تمامًا، وليس مسموحًا لك» — رمز صحيح لكن الدور غير كافٍ. خلط الاثنين خطأً متكرر جدًا في المقابلات.

```

async refresh(refreshToken: string) {
  let payload: JwtPayload;
  try {
    payload = await this.jwt.verifyAsync(refreshToken, {
      secret: this.config.get("JWT_REFRESH_SECRET"),
    });
  } catch {
    throw new UnauthorizedException("Invalid refresh token");
  }

  ابحث عن رمز حيّ لهذا المستخدم يطابق التجزئة
  const rows = await this.prisma.refreshToken.findMany({
    where: { userId: payload.sub, revokedAt: null, expiresAt: { gt: new Date() } },
  });

  let matched = null;
  for (const row of rows) {
    if (await bcrypt.compare(refreshToken, row.tokenHash)) { matched = row; break; }
  }

  if (!matched) {
    رمز صالح التوقيع لكنه مُبطل ← على الأرجح مسروق ومُعاد استعماله
    await this.prisma.refreshToken.updateMany({
      where: { userId: payload.sub, revokedAt: null },
      data: { revokedAt: new Date() },      أبطل كل جلسات هذا المستخدم
    });
    throw new UnauthorizedException("Refresh token reuse detected");
  }

  التدوير: أبطل المستعمل وأصدر زوجًا جديدًا
  await this.prisma.refreshToken.update({
    where: { id: matched.id },
    data: { revokedAt: new Date() },
  });

  return this.issueTokens(payload.sub, payload.email, payload.role);
}

async logout(refreshToken: string) {
  تمرين لك: أبطل هذا الرمز تحديدًا (نفس منطق البحث أعلاه)
}

```

لماذا حلقة مقارنة بدل بحث مباشر؟

لأننا خزّنا تجزئة الرمز، والتجزئة غير حتميّة (ملح مختلف كل مرّة) فلا يمكن البحث بها في **WHERE**. مع مئات الجلسات لكل مستخدم يصير هذا بطيئًا؛ الحل العملي: أضف **jti** (معرف رمز) إلى الحمولة وخزّنه بجانب التجزئة، فتبحث بالمعرف وتقران التجزئة مرّة واحدة.

خطوة ٤٥ لا تُسرّب كلمة السر أبدًا

خطوة صغيرة، وهي أوّل ما يفحصه المُراجع.

القاعدة الأبسط والأضمن: لا تُرجع كائن المستخدم كاملاً. استعمل `select` صريحًا دائمًا:

```
const user = await this.prisma.user.findUnique({
  where: { id },
  select: { id: true, email: true, name: true, role: true, createdAt: true },
  password غير مذكور ← لا يخرج أبدًا
});
```

تحقق بنفسك

```
curl http://localhost:3000/api/auth/me -H "Authorization: Bearer <TOKEN>" | grep -i password
```

يجب ألا يطبع شيئًا. كرّر هذا الفحص على كل نقطة نهاية تعيد مستخدمًا.

خطوة ٤٦ جرّب دورة المصادقة كاملة

سيع تجارب — لا تتخطّ واحدة.

api/requests.http

تسجيل 1)

```
POST {{host}}/auth/register
Content-Type: application/json
```

```
{ "email": "sara@test.com", "name": "Sara", "password": "Passw0rd!" }
```

دخول — انسخ accessToken من الاستجابة 2)

```
POST {{host}}/auth/login
Content-Type: application/json
```

```
{ "email": "sara@test.com", "password": "Passw0rd!" }
```

@token = ضع_الرمز_هنا

بياناتي 3)

```
GET {{host}}/auth/me
Authorization: Bearer {{token}}
```

إنشاء منتج بمستخدم عادي — يجب 403 4)

```
POST {{host}}/products
Authorization: Bearer {{token}}
Content-Type: application/json
```

```
{ "slug": "test-item", "price": 10, "categoryId": 1,
  "translations": [ { "locale": "en", "name": "Test", "description": "A test product" } ] }
```

قائمة التحقق

الحالة	المتوقع
تسجيل بريد جديد	201 + رمزان
تسجيل بنفس البريد	409
دخول بكلمة سر خاطئة	401 Invalid credentials
دخول بريد غير موجود	401 — نفس الرسالة بالضبط
/auth/me بلا ترويسة	401
/auth/me برمز مُعدّل حرفًا	401 — التوقيع لم يعد صحيحًا
إنشاء منتج بحساب USER	403
إنشاء منتج بحساب admin@minishop.test	201
refresh بالرمز نفسه مرّتين	الأولى تنجح، الثانية 401 reuse detected

انظر داخل الرمز

الصق رمز الوصول في jwt.io. ستري ثلاثة أجزاء مفصولة بنقاط، والحمولة الوسطى **مقروءة تمامًا** بلا مفتاح. الدرس: **JWT موقع لا مشفّر**. التوقيع يمنع التزوير ولا يمنع القراءة. لا تضع في الحمولة شيئًا لا تقبل أن يراه صاحب الرمز.

تمرين المرحلة 6

1. أكمل دالة `logout` بحيث تُبطل رمز التجديد المُرسَل فقط. اختبر أنّ `refresh` بعده يعطي `401`.
2. أضف `POST /auth/change-password`: يطلب كلمة السر القديمة، ويُبطل كل رموز التجديد بعد التغيير. لماذا الإبطال ضروري؟
3. أنقص `JWT_ACCESS_TTL` إلى `30s`. سجّل الدخول، انتظر دقيقة، ثم اطلب `/auth/me`. اقرأ رسالة الخطأ بدقّة.
4. أضف مالكية: اجعل `PATCH /products/:id` مسموحًا لمن دوره `ADMIN` أو من أنشأ المنتج. (تلميح: حارس لا يكفي — تحتاج فحصًا داخل الخدمة يحتاج بيانات المورد).
5. ابحث عن الفرق بين `bcrypt` و `argon2`، وقرّر أيّهما تختار ولماذا. هذا سؤال يُطرح كثيرًا.

تعدد اللغات في ال API

أكثر موضوع يُساء فهمه: «الترجمة» ثلاثة أشياء مختلفة تمامًا، لكل واحد حلّ مختلف.

خطوة ٤٧ افصل الطبقات الثلاث في ذهنك أولًا

قبل أي كود — هذا الجدول هو نصف الحل.

الطبقة	مثال	أين تُحلّ
نصوص الواجهة	«أضف إلى السلة»، «تسجيل الدخول»	في <code>Next.js</code> عبر <code>next-intl</code> — لا تمرّ بال API إطلاقًا
رسائل ال API	«الصنف غير موجود»، «البريد مسجّل مسبقًا»	في <code>NestJS</code> عبر <code>nestjs-i18n</code> حسب ترويسة <code>Accept-Language</code>
محتوى قاعدة البيانات	اسم المنتج ووصفه	في المخطّط: جدول ترجمات — بناءه المستخدم، لا المبرمج

الخطأ الشائع

محاولة حلّ الطبقات الثلاث بأداة واحدة. النتيجة إمّا API يعيد نصوص واجهة (فتصير الواجهة عاجزة عن تغيير كلمة بلا نشر خلفي)، أو ملفات ترجمة ضخمة فيها أسماء منتجات (فيصير إضافة منتج يتطلّب نشرًا). **افصلها من اليوم الأول** — وهذا بالضبط ما يُسأل عنه حين يقولون «كيف تدعم لغتين؟».

خطوة ٤٨ ركبّ `nestjs-i18n`

لرسائل ال API فقط.

```
npm i nestjs-i18n
```

```
api/src/i18n/en/messages.json
```

```
{
  "product": {
    "not_found": "Product {id} was not found",
    "created": "Product created successfully"
  },
  "category": {
    "not_found": "Category {id} was not found",
    "has_products": "Cannot delete a category that has {count} product(s)"
  },
  "auth": {
    "invalid_credentials": "Invalid email or password",
    "email_taken": "This email is already registered",
    "forbidden": "You do not have permission to perform this action"
  }
}
```

api/src/i18n/ar/messages.json

```
{
  "product": {
    "not_found": "المنتج {id} غير موجود",
    "created": "تم إنشاء المنتج بنجاح"
  },
  "category": {
    "not_found": "الصنف {id} غير موجود",
    "has_products": "لا يمكن حذف صنف يحتوي على {count} منتج"
  },
  "auth": {
    "invalid_credentials": "البريد الإلكتروني أو كلمة السر غير صحيحة",
    "email_taken": "هذا البريد مسجل مسبقًا",
    "forbidden": "لا تملك صلاحية تنفيذ هذا الإجراء"
  }
}
```

ملاحظة: النصوص العربية هنا معروضة بلون التعليق لتظهر بشكل صحيح في هذه الصفحة؛ في ملفك الفعلي هي قيم نصية عادية بين علامتي اقتباس.

api/src/app.module.ts

```
import { AcceptLanguageResolver, I18nModule, QueryResolver } from "nestjs-i18n";
import { join } from "path";

I18nModule.forRoot({
  fallbackLanguage: "en",
  loaderOptions: {
    path: join(__dirname, "/i18n/"),
    watch: true,
    // أعد التحميل عند تعديل ملفات الترجمة
  },
  resolvers: [
    { use: QueryResolver, options: ["lang"] }, // مفيد للتجربة — ?lang=ar
    AcceptLanguageResolver, // ترويسة المتصفح — الطريقة الصحيحة
  ],
}),
```

ملفات JSON لا تُنسخ إلى مجلد البناء تلقائيًا

بعد `npm run build` ستجد `dist/` بلا مجلد `i18n`، فيفشل التطبيق في الإنتاج فقط. الحل في `nest-cli.json`:

```
"compilerOptions": {
  "assets": [{ "include": "i18n/**/*", "watchAssets": true }]
}
```

هذا صنف كامل من الأخطاء: يعمل محليًا ويفشل بعد البناء. تذكره مع كل ملف غير `.ts`. يحتاجه تطبيقك.

```
import { I18nService, I18nContext } from "nestjs-i18n";

constructor(
  private readonly prisma: PrismaService,
  private readonly i18n: I18nService,
) {}

async findOne(id: number) {
  const product = await this.prisma.product.findUnique({ where: { id } });
  if (!product) {
    throw new NotFoundException(
      this.i18n.t("messages.product.not_found", {
        lang: I18nContext.current()?.lang, لغة الطلب الحالي
        args: { id },
      }),
    );
  }
  return product;
}
```

جربها الآن

```
curl "http://localhost:3000/api/products/999"
curl "http://localhost:3000/api/products/999" -H "Accept-Language: ar"
curl "http://localhost:3000/api/products/999?lang=ar"
```

الأولى إنجليزية، والثانية والثالثة عربية. لو بقيت إنجليزية: تأكد أن `I18nModule` مسجل، وأن اسم المفتاح يبدأ باسم الملف `(messages.)`.

وإلا بقيت نصف الأخطاء بالإنجليزية.

رسائل `class-validator` إنجليزية افتراضيًا. اربطها بملفات الترجمة عبر `i18nValidationMessage`:

```
api/src/i18n/ar/validation.json
```

```
{
  "IS_EMAIL": "صيغة البريد الإلكتروني غير صحيحة",
  "MIN_LENGTH": "يجب ألا يقلّ الحقل عن {constraints.0} حرفًا",
  "IS_NOT_EMPTY": "هذا الحقل مطلوب"
}
```

```
import { i18nValidationMessage } from "nestjs-i18n";

export class RegisterDto {
  @IsEmail({}, { message: i18nValidationMessage("validation.IS_EMAIL") })
  email!: string;

  @MinLength(8, { message: i18nValidationMessage("validation.MIN_LENGTH") })
  password!: string;
}
```

وفي `main.ts` استبدل الأنبوب والفلتر بنسختي `i18n`:

```
import { I18nValidationPipe, I18nValidationExceptionFilter } from "nestjs-i18n";

app.useGlobalPipes(new I18nValidationPipe({ whitelist: true, transform: true }));
app.useGlobalFilters(new I18nValidationExceptionFilter({ detailedErrors: false }));
```

خطوة ٥١ أعِد محتوى قاعدة البيانات بلغة الطلب

الطبقة الثالثة — وهي الأهم في مشروع حقيقي.

حتى الآن نعيد مصفوفة `translations` كاملة وندع الواجهة تختار. هذا خيار مشروع، لكن الأنظف أن يعيد الـ `API` شكلًا مسطّحًا بلغة واحدة:


```
import { Locale } from "@prisma/client";

type Translated<T> = T & { translations: { locale: Locale }[] };

يختار ترجمة اللغة المطلوبة، ويرجع للإنجليزية إن لم توجد، ثم لأي ترجمة متاحة
export function pickTranslation<T extends { locale: Locale }>(
  translations: T[],
  locale: Locale,
): T | undefined {
  return (
    translations.find((t) => t.locale === locale) ??
    translations.find((t) => t.locale === Locale.en) ??
    translations[0]
  );
}

export function localizeProduct(product: any, locale: Locale) {
  const tr = pickTranslation(product.translations, locale);
  const { translations, category, ...rest } = product;
  return {
    ...rest,
    name: tr?.name ?? product.slug,
    description: tr?.description ?? "",
    category: category
      ? {
          id: category.id,
          slug: category.slug,
          name: pickTranslation(category.translations, locale)?.name ?? category.slug,
        }
      : undefined,
  };
}
```

واستعملها في الخدمة:

```
const locale = (I18nContext.current()?.lang ?? "en") as Locale;
return { items: items.map((p) => localizeProduct(p, locale)), meta };
```

جربها الآن

```
curl "http://localhost:3000/api/products/1" -H "Accept-Language: en"
curl "http://localhost:3000/api/products/1" -H "Accept-Language: ar"
```

الشكل المتوقَّع — بلا مصفوفة ترجمات، اسم واحد جاهز للعرض:

```
{ "id": 1, "slug": "wireless-mouse", "price": "24.9",
  "name": "Wireless Mouse", "description": "Description of Wireless Mouse",
  "category": { "id": 1, "slug": "electronics", "name": "Electronics" } }
```

الاحتياط (fallback) ليس تفصيلًا ماذا لو أُضيف منتج بالعربية فقط وطلبه زائر إنجليزي؟ بلا احتياط تظهر الصفحة فارغة أو تنهار. سلسلتنا: اللغة المطلوبة → الإنجليزية → أي ترجمة موجودة → ال slug. لا تُظهر فراغًا للمستخدم أبدًا.

ملاحظة للوحة الإدارة

المدير يحتاج كل الترجمات ليعدّلها، لا لغة واحدة مسطّحة. الحل البسيط: معامل `raw=true` على مسارات الإدارة، أو مسارات إدارة منفصلة تحت `/api/admin/`. فكّر في الأمر: لكل مستهلك شكل بيانات يناسبه.

خطوة ٥٢ تحقق من كل الطبقات معًا

قائمة التحقق النهائية للمرحلة

الطلب	المتوقع
GET /products/999 ب Accept-Language: ar	رسالة خطأ عربية
GET /products/1 ب ar	اسم ووصف عربيّان
POST /auth/register بريد خاطئ + ar	رسالة تحقق عربية
بلا ترويسة لغة إطلاقًا	كل شيء بالإنجليزية (الاحتياط)
Accept-Language: fr	إنجليزية — لا انهيار

```
git add . && git commit -m "feat(i18n): API messages, validation and content localization"
```

تمرين المرحلة ٧

- أضف لغة ثالثة `tr` إلى ال `enum Locale`. كم ملفًا تحتاج تعديله؟ وكم هجرة؟ قارن هذا بما لو كنت استعملت أعمدة `name_eng` و `name_ar`.
- اجعل `localizeProduct` عامًّا (generic) بحيث تعمل مع الأصناف أيضًا بلا تكرار.
- أضف حقل `availableLocales: ["ar", "en"]` إلى استجابة المنتج، ليعرف العميل ما الترجمات الموجودة فعليًا.
- ما رأيك في إعادة اللغة في ترويسة `Content-Language` من الاستجابة؟ نفّذها.

الواجهة بـ Next.js

مسار لغة، اتجاه تلقائي، طبقة اتصال واحدة، وصفحات تجلب من الـ API الذي بنيته.

خطوة ٥٣ أنشئ مشروع Next

من جذر `mini-shop/` (وليس من داخل `api/`):

```
npx create-next-app@latest web
```

أجب على الأسئلة هكذا:

السؤال	الجواب
TypeScript?	Yes
ESLint?	Yes
Tailwind CSS?	Yes — يوفر وقتًا كثيرًا هنا
src/ directory?	Yes
App Router?	Yes — إلزامي لهذه الورشة
Turbopack?	Yes — تطوير أسرع
import alias?	الافتراضي <code>@/*</code>

```
cd web
npm run dev -- -p 3001
```

وثبتت المنفذ في `package.json` حتى لا تكتبه كل مرة:

```
"scripts": { "dev": "next dev -p 3001" }
```

جربها الآن

`http://localhost:3001` يعرض صفحة `Next` الافتراضية. الآن عندك خادمان يعملان في طرفيتين: الـ `API` على `3000` والواجهة على `3001`.

خطوة ٥٤ افهم توجيه الملفات

في Next لا ملف مسارات — المجلد هو المسار.

الملف	دوره
app/page.tsx	الصفحة على /
app/products/page.tsx	الصفحة على /products
app/products/[id]/page.tsx	مسار متغير /products/7
layout.tsx	غلاف يلف الصفحات تحته — يقابل Blade layout
loading.tsx	يُعرض تلقائيًا أثناء جلب بيانات الصفحة
error.tsx	يُعرض عند رمي خطأ — يجب أن يكون Client Component
not-found.tsx	صفحة 404
route.ts	نقطة نهاية API داخل Next نفسه
middleware.ts	يعمل قبل كل طلب — يقابل الـ middleware عندك

القاعدة الذهبية في App Router كل مكون هو **Server Component** افتراضيًا: يعمل على الخادم، يقدر يستعلم مباشرة، ولا يصل كوده إلى المتصفح. تكتب "use client" في أول الملف فقط إن احتجت `useState` أو `useEffect` أو `onClick` أو أي واجهة متصفح. ابدأ دائمًا بمكون خادم، وانزل إلى العميل عند أصغر جزء يحتاجه. هذا أكثر سؤال يُطرح عن Next اليوم.

خطوة ٥٥ ركب next-intl

مسار لغة حقيقي: .../ar/ و .../en/

```
npm i next-intl
```

```
web/src/i18n/routing.ts
```

```
import { defineRouting } from "next-intl/routing";
import { createNavigation } from "next-intl/navigation";

export const routing = defineRouting({
  locales: ["ar", "en"],
  defaultLocale: "ar",
  localePrefix: "always", // دائمًا — أوضح وأفضل للأرشفة /ar/products
});
```

بدائل مدركة للغة عن Link و useRouter — استعملها بدل الأصلية

```
export const { Link, redirect, usePathname, useRouter } = createNavigation(routing);
```

web/src/i18n/request.ts

```
import { getRequestConfig } from "next-intl/server";
import { routing } from "../routing";

export default getRequestConfig(async ({ requestLocale }) => {
  let locale = await requestLocale;
  if (!locale || !routing.locales.includes(locale as never)) {
    locale = routing.defaultLocale;
  }
  return {
    locale,
    messages: (await import(`../../messages/${locale}.json`)).default,
  };
});
```

web/next.config.ts

```
import createNextIntlPlugin from "next-intl/plugin";
const withNextIntl = createNextIntlPlugin("../src/i18n/request.ts");

const nextConfig = {
  images: {
    remotePatterns: [{ protocol: "http", hostname: "localhost", port: "3000" }],
  },
};

export default withNextIntl(nextConfig);
```

web/src/middleware.ts

```
import createMiddleware from "next-intl/middleware";
import { routing } from "../i18n/routing";

export default createMiddleware(routing);

export const config = {
  طَبِّقْ عَلَى كُلِّ شَيْءٍ عِداً لملفات النظام والملفات الساكنة
  matcher: ["/((?!api|_next|_vercel|.*\\.\\.\\.)*.*)"],
};
```

web/messages/en.json

```
{
  "nav": { "home": "Home", "products": "Products", "login": "Sign in", "admin": "Dashboard" },
  "products": {
    "title": "Products",
    "search": "Search products...",
    "empty": "No products found",
    "price": "Price",
    "addedOn": "Added on {date}",
    "showing": "Showing {from}-{to} of {total}"
  },
  "auth": { "email": "Email", "password": "Password", "submit": "Sign in" }
}
```

web/messages/ar.json

```
{
  "nav": { "home": "الرئيسية", "products": "المنتجات",
    "login": "تسجيل الدخول", "admin": "لوحة التحكم" },
  "products": {
    "title": "المنتجات",
    "search": "ابحث عن منتج...",
    "empty": "لا توجد منتجات",
    "price": "السعر",
    "addedOn": "{date} أُضيف في",
    "showing": "عرض {to}-{from} من {total}"
  }
}
```

احذف `src/app/layout.tsx` و `src/app/page.tsx`، وأنشئ بدلتهما:

```
import { NextIntlClientProvider } from "next-intl";
import { getMessages, setRequestLocale } from "next-intl/server";
import { notFound } from "next/navigation";
import { routing } from "@/i18n/routing";
import "../globals.css";

export function generateStaticParams() {
  return routing.locales.map((locale) => ({ locale }));
}

export default async function LocaleLayout({
  children,
  params,
}: {
  children: React.ReactNode;
  params: Promise<{ locale: string }>; في Next 15 صارت Promise
}) {
  const { locale } = await params;
  if (!routing.locales.includes(locale as never)) notFound();

  setRequestLocale(locale);
  const messages = await getMessages();

  return (
    <html lang={locale} dir={locale === "ar" ? "rtl" : "ltr"}>
      <body className={locale === "ar" ? "font-arabic" : ""}>
        <NextIntlClientProvider messages={messages}>
          {children}
        </NextIntlClientProvider>
      </body>
    </html>
  );
}
```

هذا السطر هو كل سّر دعم العربية

`dir={locale === "ar" ? "rtl" : "ltr"}` ومع `Tailwind` استعمل الأصناف المنطقية: `ms-4` و `me-4` (بداية/نهاية) بدل `ml-4` و `mr-4` (يسار/يمين)، و `text-start` بدل `text-left`. عندها ينقلب التخطيط كلّ صحيحًا بتغيير سمة واحدة، بلا أي `CSS` إضافي.

تغيير مهم في Next 15

`params` و `searchParams` صارتا `Promise` ويجب `await` قبل الاستعمال. كل الشروحات القديمة على الإنترنت تصل إليهما مباشرة — وستحصل على خطأ محيّر إن قلّدتها.

web/src/components/LocaleSwitcher.tsx

```
"use client";

import { usePathname, useRouter } from "@i18n/routing";
import { useLocale } from "next-intl";

export default function LocaleSwitcher() {
  const locale = useLocale();
  const pathname = usePathname(); // المسار بلا بادئة اللغة
  const router = useRouter();
  const other = locale === "ar" ? "en" : "ar";

  return (
    <button
      onClick={() => router.replace(pathname, { locale: other })}
      className="rounded border px-3 py-1 text-sm"
    >
      {other === "ar" ? "العربية" : "English"}
    </button>
  );
}
```

جربها الآن

افتح <http://localhost:3001> — يجب أن يحوّلك تلقائيًا إلى `/ar`. اضغط الزر: يصير `/en`، ويتغيّر اتجاه الصفحة كلّها من يمين-يسار إلى يسار-يمين. افحص `<html>` في أدوات المطوّر: سمة `dir` تغيّرت فعلًا.

web/.env.local

```
NEXT_PUBLIC_API_URL=http://localhost:3000/api
API_URL_INTERNAL=http://localhost:3000/api
```

افهم البادئة `NEXT_PUBLIC_` جيّدًا

أي متغيّر يبدأ بها يُحقن حرفيًا في حزمة المتصفح ويستطيع أي زائر قراءته من مصدر الصفحة. استعملها للعناوين العامة فقط. أي سرّ (مفتاح API، سرّ JWT، رابط قاعدة بيانات) يجب أن يكون بدونها ليبقى على الخادم. هذا سؤال مقابلة، وتسريبه خطأ حقيقي يقع فيه كثيرون.


```

const BASE = process.env.API_URL_INTERNAL ?? process.env.NEXT_PUBLIC_API_URL!;

export class ApiError extends Error {
  constructor(public status: number, message: string) {
    super(message);
  }
}

type Options = RequestInit & { locale?: string; token?: string };

export async function api<T>(path: string, options: Options = {}): Promise<T> {
  const { locale, token, headers, ...rest } = options;

  const res = await fetch(`${BASE}${path}`, {
    ...rest,
    headers: {
      "Content-Type": "application/json",
      ...(locale ? { "Accept-Language": locale } : {}),
      ...(token ? { Authorization: `Bearer ${token}` } : {}),
      ...headers,
    },
  });

  if (res.status === 204) return undefined as T; لا محتوى — لا تحاول قراءة JSON

  const body = await res.json().catch(() => ({}));

  if (!res.ok) {
    const message = Array.isArray(body.message)
      ? body.message.join(", ") أخطاء التحقق تأتي مصفوفة
      : body.message ?? `Request failed (${res.status})`;
    throw new ApiError(res.status, message);
  }

  return body as T;
}

```

لماذا طبقة واحدة عنوان الأساس، ترويسة اللغة، الرمز، تحويل الخطأ إلى استثناء، ومعالجة 204 — خمسة أشياء لو تكررت في عشرين مكانًا فستنسأها في واحد منها. وحين تغيّر شيئًا (تضيف ترويسة تتبّع مثلًا) تعدّل ملفًا واحدًا.

web/src/app/[locale]/products/page.tsx

```
import { getTranslations, setRequestLocale } from "next-intl/server";
import { api } from "@/lib/api";
import { Link } from "@/i18n/routing";
import ProductCard from "@/components/ProductCard";
import SearchBox from "@/components/SearchBox";

type Product = {
  id: number; slug: string; price: string; imageUrl: string | null;
  name: string; description: string;
  category: { slug: string; name: string };
};

type ListResponse = {
  items: Product[];
  meta: { total: number; page: number; limit: number; pages: number };
};

export default async function ProductsPage({
  params,
  searchParams,
}: {
  params: Promise<{ locale: string }>;
  searchParams: Promise<{ page?: string; search?: string; sort?: string }>;
}) {
  const { locale } = await params;
  const sp = await searchParams;
  setRequestLocale(locale);
  const t = await getTranslations("products");

  const qs = new URLSearchParams({
    page: sp.page ?? "1",
    limit: "12",
    published: "true",
    ...(sp.search ? { search: sp.search } : {}),
    ...(sp.sort ? { sort: sp.sort } : {}),
  });

  const data = await api<ListResponse>(`/products?${qs}`, {
    locale,
    cache: "no-store",
    // اجلب طازجاً في كل طلب — نقاشها بعد قليل
  });

  return (
    <main className="mx-auto max-w-5xl p-6">
      <h1 className="mb-4 text-2xl font-bold">{t("title")}</h1>
      <SearchBox placeholder={t("search")} />

      {data.items.length === 0 ? (
        <p className="mt-8 text-gray-500">{t("empty")}</p>
      ) : (
        <div className="mt-6 grid grid-cols-1 gap-4 sm:grid-cols-2 lg:grid-cols-3">

```

```

      {data.items.map((p) => (
        <ProductCard key={p.id} product={p} locale={locale} />
      ))}
    </div>
  )}

  <nav className="mt-8 flex gap-2">
    {Array.from({ length: data.meta.pages }, (_, i) => i + 1).map((n) => (
      <Link
        key={n}
        href={{ pathname: "/products", query: { ...sp, page: n } }}
        className={n === data.meta.page ? "font-bold underline" : ""}
      >
        {n}
      </Link>
    ))}
  </nav>
</main>
);
}

```

وبطاقة المنتج — مكوّن خادم أيضًا، لأنّه لا يحتاج أي تفاعل:

web/src/components/ProductCard.tsx

```

import Image from "next/image";
import { Link } from "@/i18n/routing";

export default function ProductCard({ product, locale }: Props) {
  const price = new Intl.NumberFormat(locale, {
    style: "currency",
    currency: "USD",
  }).format(Number(product.price)); تنسيق مدرك للغة — مجاني

  return (
    <Link href={` /products/${product.id}`} className="rounded border p-4 hover:shadow">
      {product.imageUrl && (
        <Image
          src={`http://localhost:3000${product.imageUrl}`}
          alt={product.name} width={300} height={200}
          className="mb-3 rounded object-cover"
        />
      )}
      <h2 className="font-semibold">{product.name}</h2>
      <p className="text-sm text-gray-500">{product.category.name}</p>
      <p className="mt-2 font-bold">{price}</p>
    </Link>
  );
}

```

جربها الآن

`http://localhost:3001/ar/products` — يجب أن ترى منتجاتك بأسمائها العربية. بدّل إلى `/en/products` — نفس المنتجات بالإنجليزية، والأسعار بتنسيق مختلف.

افتح مصدر الصفحة (`Ctrl+U`): ستجد أسماء المنتجات داخل الـ **HTML نفسه**. هذا معنى التصيير على الخادم — محرّكات البحث تراها، والمستخدم يرى المحتوى قبل تحميل أي `JavaScript`.

إن ظهر `ECONNREFUSED`

خادم الـ `API` متوقّف. ولاحظ الفرق: هذا الجلب يحدث في خادم **Next** لا في المتصفّح، فلا علاقة له بـ `CORS` إطلاقًا. لهذا استعملنا `API_URL_INTERNAL` هنا.

```
web/src/app/[locale]/products/[id]/page.tsx
```

```
import { notFound } from "next/navigation";
import type { Metadata } from "next";
import { api, ApiError } from "@lib/api";

type Props = { params: Promise<{ locale: string; id: string }> };

async function getProduct(id: string, locale: string) {
  try {
    return await api<Product>(`/products/${id}`, { locale, next: { revalidate: 60 } });
  } catch (e) {
    if (e instanceof ApiError && e.status === 404) return null;
    throw e;
    // أي خطأ آخر ← error.tsx
  }
}

// عنوان ووصف مختلفان لكل لغة — مهمّان لمحركات البحث
export async function generateMetadata({ params }: Props): Promise<Metadata> {
  const { id, locale } = await params;
  const product = await getProduct(id, locale);
  if (!product) return { title: "Not found" };

  return {
    title: `${product.name} | mini-shop`,
    description: product.description.slice(0, 160),
    openGraph: { title: product.name, images: product.imageUrl ? [product.imageUrl] : [] },
  };
}

export default async function ProductPage({ params }: Props) {
  const { id, locale } = await params;
  const product = await getProduct(id, locale);
  if (!product) notFound(); // يعرض not-found.tsx ويرسل 404 حقيقياً ←

  return (
    <article className="mx-auto max-w-3xl p-6">
      <h1 className="text-3xl font-bold">{product.name}</h1>
      <p className="mt-4 leading-8">{product.description}</p>
    </article>
  );
}
```

ثلاث كلمات تحدّد سلوك التصيير كلّ `cache: "no-store"` → اجلب في كل طلب (SSR) — للبيانات المتغيّرة أو الخاصة بالمستخدم.

بلا شيء → يُولّد وقت البناء (SSG) — للمحتوى الثابت. `next: { revalidate: 60 }` → ولّد مرّة وأعدّ التوليد كل ٦٠ ثانية (ISR) — الخيار الأمثل لصفحات المنتجات.

افهم الثلاثة؛ فسؤال «ما الفرق بين SSR وSSG وISR ومتى تستعمل كلّاً منها؟» شبه مؤكّد.

web/src/app/[locale]/products/loading.tsx

```
export default function Loading() {
  return (
    <div className="mx-auto max-w-5xl p-6">
      <div className="mb-6 h-8 w-40 animate-pulse rounded bg-gray-200" />
      <div className="grid grid-cols-1 gap-4 sm:grid-cols-2 lg:grid-cols-3">
        {Array.from({ length: 6 }).map((_, i) => (
          <div key={i} className="h-48 animate-pulse rounded bg-gray-100" />
        ))}
      </div>
    </div>
  );
}
```

web/src/app/[locale]/products/error.tsx

```
"use client";
إلزامي — حدود الخطأ تحتاج العميل

export default function Error({
  error, reset,
}: { error: Error; reset: () => void }) {
  return (
    <div className="mx-auto max-w-md p-8 text-center">
      <h2 className="text-xl font-bold">Something went wrong</h2>
      <p className="mt-2 text-sm text-gray-500">{error.message}</p>
      <button onClick={reset} className="mt-4 rounded bg-black px-4 py-2 text-white">
        Try again
      </button>
    </div>
  );
}
```

جربها الآن

أوقف خادم ال API (Ctrl+C في طرفيته) ثم حدّث صفحة المنتجات. يجب أن ترى صفحة الخطأ لا انهيارًا أبيض. أعد تشغيل ال API واضغط Try again.

ولرؤية loading.tsx: أضف تأخيرًا مؤقتًا في ال API (await new Promise(r => setTimeout(r, 2000))) في الخدمة وحدّث الصفحة.

web/src/components/SearchBox.tsx

```
"use client";

import { useRouter, usePathname } from "@/i18n/routing";
import { useSearchParams } from "next/navigation";
import { useState, useTransition } from "react";

export default function SearchBox({ placeholder }: { placeholder: string }) {
  const router = useRouter();
  const pathname = usePathname();
  const searchParams = useSearchParams();
  const [value, setValue] = useState(searchParams.get("search") ?? "");
  const [isPending, startTransition] = useTransition();

  function submit(e: React.FormEvent) {
    e.preventDefault();
    const params = new URLSearchParams(searchParams);
    value ? params.set("search", value) : params.delete("search");
    params.delete("page"); بحث جديد ← ارجع للصفحة الأولى
    startTransition(() => router.push(`${pathname}?${params}`));
  }

  return (
    <form onSubmit={submit} className="flex gap-2">
      <input
        value={value}
        onChange={(e) => setValue(e.target.value)}
        placeholder={placeholder}
        className="flex-1 rounded border px-3 py-2"
      />
      <button disabled={isPending} className="rounded bg-black px-4 py-2 text-white">
        {isPending ? "..." : "Search"}
      </button>
    </form>
  );
}
```

النمط الذي يجب أن تتقنه

الحالة تعيش في **الرابط** لا في **useState**. الفائدة: الصفحة قابلة للمشاركة، وزرّ الرجوع يعمل، والخادم يعيد التصيير بالبيانات الصحيحة تلقائيًا. وهذا هو الفرق العملي بين تطبيق **React** تقليدي وتطبيق **Next** مبنيّ بشكل صحيح.

تمرين المرحلة ٨

1. أضف قائمة منسدلة للترتيب (`sort`) تعمل بنفس نمط صندوق البحث.
2. أضف فلتر أصناف: اجلب الأصناف في مكوّن الخادم ومزّرها كأزرار تعدّل `?categoryId=`.
3. أضف شريط تنقل مشتركًا في `layout.tsx` فيه الروابط ومبدّل اللغة. استعمل `Link` من `@/i18n/routing` لا من `next/link` — جرّب الاثنين وانظر الفرق في الرابط الناتج.
4. غيّر `cache: "no-store"` إلى `next: { revalidate: 30 }` في صفحة القائمة. عدّل منتجًا من `Prisma Studio` وراقب متى يظهر التغيير.
5. أضف `not-found.tsx` مترجمًا بلغتين.

الدخول ولوحة الإدارة

ربط المصادقة التي بنيتها بالواجهة، بطريقة آمنة لا تضع الرمز في متناول أي سكربت.

أين نخزن الرمز؟ ثلاثة خيارات وقرار واحد

المكان	الخطر	الحكم
localStorage	أي سكربت في الصفحة يقرأه — ثغرة XSS واحدة تكفي لسرقة الجلسة	شائع، وسيئ
كوكبي عادية	JavaScript يقرأها أيضًا — نفس المشكلة	لا
كوكبي httpOnly	CSRF — ويُعالج بـ SameSite=Lax	اختيارنا

لماذا هذا ممكن أصلاً في Next لأنّ عندنا خادمًا. المتصفح لا يلمس الرمز إطلاقًا: يضعه خادم Next في كوكبي httpOnly، ويقرأه من الكوكبي عند كل جلب، ويرسله في ترويسة Authorization إلى الـ API. في تطبيق React صرف (بلا خادم) هذا غير متاح، ولهذا يلجأ الناس إلى localStorage. هذه المقارنة بالذات جواب ممتاز في المقابلة.

web/src/lib/session.ts

```
import { cookies } from "next/headers";
import { redirect } from "@/i18n/routing";

const ACCESS = "access_token";
const REFRESH = "refresh_token";

export async function setSession(accessToken: string, refreshToken: string) {
  const jar = await cookies(); // في Next 15 صارت غير متزامنة
  const common = {
    httpOnly: true, // لا يراها إطلافاً JavaScript
    secure: process.env.NODE_ENV === "production",
    sameSite: "lax" as const, // حماية أساسية من CSRF
    path: "/",
  };
  jar.set(ACCESS, accessToken, { ...common, maxAge: 60 * 15 });
  jar.set(REFRESH, refreshToken, { ...common, maxAge: 60 * 60 * 24 * 7 });
}

export async function getToken() {
  return (await cookies()).get(ACCESS)?.value ?? null;
}

export async function clearSession() {
  const jar = await cookies();
  jar.delete(ACCESS);
  jar.delete(REFRESH);
}

export async function getCurrentUser(locale: string) {
  const token = await getToken();
  if (!token) return null;
  try {
    return await api<AuthUser>("/auth/me", { token, locale, cache: "no-store" });
  } catch {
    return null; // رمز منتج أو فاسد ← زائر
  }
}
```

web/src/app/[locale]/login/actions.ts

```
"use server"; كل ما في هذا الملف يعمل على الخادم فقط

import { api, ApiError } from "@lib/api";
import { setSession } from "@lib/session";
import { redirect } from "@i18n/routing";

export type LoginState = { error?: string };

export async function loginAction(
  _prev: LoginState,
  formData: FormData,
): Promise<LoginState> {
  const email = String(formData.get("email") ?? "");
  const password = String(formData.get("password") ?? "");
  const locale = String(formData.get("locale") ?? "ar");

  try {
    const tokens = await api<{ accessToken: string; refreshToken: string }>(
      "/auth/login",
      { method: "POST", body: JSON.stringify({ email, password }), locale },
    );
    await setSession(tokens.accessToken, tokens.refreshToken);
  } catch (e) {
    if (e instanceof ApiError) return { error: e.message };
    throw e;
  }

  redirect({ href: "/admin/products", locale }); خارج try — يرمي داخليًا
}
```

```

"use client";

import { useActionState } from "react";
import { useFormStatus } from "react-dom";
import { loginAction } from "../actions";

function SubmitButton({ label }: { label: string }) {
  const { pending } = useFormStatus(); يعرف حالة النموذج الأب
  return (
    <button disabled={pending} className="w-full rounded bg-black py-2 text-white">
      {pending ? "..." : label}
    </button>
  );
}

export default function LoginForm({ locale, labels }: Props) {
  const [state, formAction] = useActionState(loginAction, {});

  return (
    <form action={formAction} className="mx-auto mt-16 max-w-sm space-y-3">
      <input type="hidden" name="locale" value={locale} />

      <input name="email" type="email" required
        placeholder={labels.email}
        className="w-full rounded border px-3 py-2" />

      <input name="password" type="password" required
        placeholder={labels.password}
        className="w-full rounded border px-3 py-2" />

      {state.error && (
        <p className="rounded bg-red-50 p-2 text-sm text-red-700">{state.error}</p>
      )}

      <SubmitButton label={labels.submit} />
    </form>
  );
}

```

لماذا Server Action بدل fetch من العميل

كلمة السر لا تمرّ عبر JavaScript في المتصفح، والرمز لا يصل إليه أبدًا، والنموذج يعمل قبل أن تُحمّل أي حزمة (تحسين تدريجي حقيقي). والتمن: لا تستطيع تصييرًا ساكنًا لهذه الصفحة — وهو ثمن مقبول لصفحة دخول.

جربها الآن

افتح `/ar/login`، وادخل بـ `Admin@12345 / admin@minishop.test`.

ثم افتح أدوات المطور ← `Application` ← `Cookies`: ستري `access_token` وبجانبه علامة `HttpOnly`. اكتب في وحدة التحكم `document.cookie` — لن يظهر الرمز. هذا هو المطلوب بالضبط.

ثم جرب كلمة سر خاطئة: يجب أن تظهر رسالة الخطأ من الـ `API` داخل النموذج، وبالعربية إن كنت على `/ar`.

خطوة ٦٥ احم مسارات الإدارة

طبقتان: `middleware` سريع، وفحص حقيقي في الصفحة.

`web/src/middleware.ts`

```
import createMiddleware from "next-intl/middleware";
import { NextRequest, NextResponse } from "next/server";
import { routing } from "../i18n/routing";

const intlMiddleware = createMiddleware(routing);

export default function middleware(req: NextRequest) {
  const { pathname } = req.nextUrl;

  /ar/admin/... أو /en/admin/...
  const isAdmin = /^\/(ar|en)\/admin(\/|$)/.test(pathname);

  if (isAdmin && !req.cookies.get("access_token")) {
    const locale = pathname.split("/")[1];
    const url = new URL(`/${locale}/login`, req.url);
    url.searchParams.set("next", pathname); أعدّه إلى وجهته بعد الدخول
    return NextResponse.redirect(url);
  }

  return intlMiddleware(req);
}

export const config = { matcher: ["/((?!api|_next|_vercel|.*\\.\\.\\.).*)" ] };
```

الـ `middleware` ليس أمانًا

هو يفحص وجود الكوكي فقط، لا صحتها ولا الدور. كوكي مزيفة تمرّ منه. الأمان الحقيقي في مكانين: الـ `API` (الحارس والدور — بنيناها في المرحلة ٦)، وفحص إضافي في مكوّن الخادم قبل عرض أي شيء حسّاس. الـ `middleware` تحسين تجربة لا حاجز. قل هذا في المقابلة — يميّزك فورًا.

web/src/app/[locale]/admin/layout.tsx

```
import { redirect } from "@/i18n/routing";
import { getCurrentUser } from "@/lib/session";

export default async function AdminLayout({ children, params }: Props) {
  const { locale } = await params;
  const user = await getCurrentUser(locale);

  الفحص الحقيقي: رمز صالح فعليًا + دور كافٍ
  if (!user) redirect({ href: "/login", locale });
  if (user.role !== "ADMIN") redirect({ href: "/", locale });

  return (
    <div className="mx-auto max-w-5xl p-6">
      <header className="mb-6 flex items-center justify-between border-b pb-3">
        <h1 className="text-xl font-bold">Dashboard</h1>
        <span className="text-sm text-gray-500">{user.email}</span>
      </header>
      {children}
    </div>
  );
}
```

الخروج **خطوة ٦٦**

web/src/app/[locale]/admin/logout-action.ts

```
"use server";

import { api } from "@/lib/api";
import { cookies } from "next/headers";
import { clearSession } from "@/lib/session";
import { redirect } from "@/i18n/routing";

export async function logoutAction(locale: string) {
  const refreshToken = (await cookies()).get("refresh_token")?.value;

  if (refreshToken) {
    أبطله في الخادم أيضًا — لا يكفي مسح الكوكبي
    await api("/auth/logout", {
      method: "POST",
      body: JSON.stringify({ refreshToken }),
    }).catch(() => {});
  }

  await clearSession();
  redirect({ href: "/login", locale });
}
```

خروج ناقص = ثغرة

مسح الكوكبي يخفي الرمز عن هذا المتصفح فقط؛ الرمز نفسه يبقى صالحًا حتى ينتهي. الخروج الصحيح يُبطل رمز التجديد في قاعدة البيانات. هذا سؤال يُطرح بصيغة: «كيف تُسجّل خروج مستخدم فورًا مع أن JWT عديم الحالة؟».

```
web/src/app/[locale]/admin/products/page.tsx
```

```
import { api } from "@lib/api";
import { getToken } from "@lib/session";
import { Link } from "@i18n/routing";
import DeleteButton from "../DeleteButton";

export default async function AdminProducts({ params }: Props) {
  const { locale } = await params;
  const token = await getToken();

  const data = await api<ListResponse>("/products?limit=100&raw=true", {
    token, locale, cache: "no-store", لوحة إدارة ← لا تخزين مؤقت أبدًا
  });

  return (
    <>
      <Link href="/admin/products/new" className="rounded bg-black px-4 py-2 text-white">
        + New product
      </Link>

      <table className="mt-4 w-full text-start text-sm">
        <thead><tr className="border-b">
          <th className="p-2 text-start">ID</th>
          <th className="p-2 text-start">Slug</th>
          <th className="p-2 text-start">Price</th>
          <th className="p-2 text-start">Published</th>
          <th></th>
        </tr></thead>
        <tbody>
          {data.items.map((p) => (
            <tr key={p.id} className="border-b">
              <td className="p-2">{p.id}</td>
              <td className="p-2">{p.slug}</td>
              <td className="p-2">{p.price}</td>
              <td className="p-2">{p.published ? "Yes" : "No"}</td>
              <td className="p-2 text-end">
                <Link href={` /admin/products/${p.id}`} className="me-3 underline">Edit</Link>
                <DeleteButton id={p.id} locale={locale} />
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </>
  );
}
```


web/src/app/[locale]/admin/products/actions.ts

```
"use server";

import { revalidatePath } from "next/cache";
import { api, ApiError } from "@lib/api";
import { getToken } from "@lib/session";
import { redirect } from "@i18n/routing";

export type FormState = { error?: string };

export async function createProduct(
  _prev: FormState,
  formData: FormData,
): Promise<FormState> {
  const locale = String(formData.get("locale"));
  const token = await getToken();

  const payload = {
    slug: String(formData.get("slug")),
    price: Number(formData.get("price")),
    stock: Number(formData.get("stock")) ?? 0,
    categoryId: Number(formData.get("categoryId")),
    published: formData.get("published") === "on",
    translations: [
      { locale: "ar",
        name: String(formData.get("name_ar")),
        description: String(formData.get("desc_ar")) },
      { locale: "en",
        name: String(formData.get("name_en")),
        description: String(formData.get("desc_en")) },
    ],
  };

  try {
    await api("/products", { method: "POST", body: JSON.stringify(payload), token, locale });
  } catch (e) {
    if (e instanceof ApiError) return { error: e.message };
    throw e;
  }

  امسح الذاكرة المؤقتة للصفحات المتأثرة — وإلا بقيت القائمة قديمة
  revalidatePath(`/${locale}/admin/products`);
  revalidatePath(`/${locale}/products`);

  redirect({ href: "/admin/products", locale });
}
```

```

"use client";

import { useActionState } from "react";
import { createProduct } from "../actions";

export default function NewProduct({ params }: { params: { locale: string } }) {
  const [state, action] = useActionState(createProduct, {});

  return (
    <form action={action} className="mt-6 max-w-xl space-y-3">
      <input type="hidden" name="locale" value={params.locale} />

      <input name="slug" required placeholder="slug (e.g. blue-mug)"
        className="w-full rounded border px-3 py-2" />

      <div className="flex gap-3">
        <input name="price" type="number" step="0.01" required placeholder="Price"
          className="w-1/2 rounded border px-3 py-2" />
        <input name="stock" type="number" placeholder="Stock"
          className="w-1/2 rounded border px-3 py-2" />
      </div>

      <input name="categoryId" type="number" required placeholder="Category ID"
        className="w-full rounded border px-3 py-2" />

      <fieldset className="rounded border p-3">
        <legend className="px-1 text-sm">العربية</legend>
        <input name="name_ar" required placeholder="الاسم"
          className="mb-2 w-full rounded border px-3 py-2" dir="rtl" />
        <textarea name="desc_ar" required placeholder="الوصف"
          className="w-full rounded border px-3 py-2" dir="rtl" />
      </fieldset>

      <fieldset className="rounded border p-3">
        <legend className="px-1 text-sm">English</legend>
        <input name="name_en" required placeholder="Name"
          className="mb-2 w-full rounded border px-3 py-2" dir="ltr" />
        <textarea name="desc_en" required placeholder="Description"
          className="w-full rounded border px-3 py-2" dir="ltr" />
      </fieldset>

      <label className="flex items-center gap-2">
        <input type="checkbox" name="published" /> Published
      </label>

      {state.error && (
        <p className="rounded bg-red-50 p-2 text-sm text-red-700">{state.error}</p>
      )}

      <button className="rounded bg-black px-4 py-2 text-white">Create</button>
    </form>
  );
}

```

جربها الآن

1. أنشئ منتجًا جديدًا من اللوحة. يجب أن تعود إلى القائمة وتراه فورًا.
2. افتح `/ar/products` في تبويب آخر — يظهر المنتج (بفضل `revalidatePath`).
3. أرسل `slug` فيه مسافات — يجب أن تظهر رسالة تحقق الـ `API` داخل النموذج.
4. احذف الترجمة الإنجليزية واحفظ — يجب أن يرفض الـ `API`. لاحظ أن الحماية في الخادم لا في النموذج.

`revalidatePath` مكونات الخادم مخزنة مؤقتًا. بعد أي كتابة يجب أن تخبر `Next` بما بطل. نسيانه ينتج أشهر عطل في `App Router`: «أنشأت العنصر لكنه لا يظهر إلّا بعد إعادة تشغيل الخادم». وله شقيق `revalidateTag` إن كنت تعلم طلبات الجلب بوسوم.

الحذف بتأكيد خطوة ٦٩

`web/src/app/[locale]/admin/products/DeleteButton.tsx`

```
"use client";

import { useTransition } from "react";
import { deleteProduct } from "../actions";

export default function DeleteButton({ id, locale }: { id: number; locale: string }) {
  const [pending, start] = useTransition();

  return (
    <button
      disabled={pending}
      onClick={() => {
        if (!confirm("Delete this product?")) return;
        start(() => { void deleteProduct(id, locale); });
      }}
      className="text-red-600 underline disabled:opacity-50"
    >
      {pending ? "..." : "Delete"}
    </button>
  );
}
```

و `deleteProduct` في `actions.ts` يستدعي `DELETE /products/:id` ثم `revalidatePath`.

اختبار القبول

الخطوة	المتوقع
افتح /	تحويل إلى /ar
/ar/products	منتجات بأسماء عربية، تخطيط من اليمين
مبدّل اللغة	نفس الصفحة إنجليزية LTR
ابحث عن mouse	الرابط يتغيّر، والنتائج تُفلتر
/ar/admin/products كزائر	تحويل إلى /ar/login
دخول بحساب USER	تحويل إلى الرئيسية — ليس مديرًا
دخول بحساب ADMIN	اللوحة تُفتح
إنشاء منتج	يظهر في اللوحة وفي الصفحة العامة
خروج ثم رجوع بزرّ المتصفّح	تحويل إلى الدخول — لا محتوى محميّ من الذاكرة

```
git add . && git commit -m "feat(web): auth with httpOnly cookies and admin dashboard"
```

تمرين المرحلة ٩

- أضف صفحة تعديل منتج `/admin/products/[id]` تملأ النموذج بالقيم الحالية. (تلميح: مكوّن خادم يجلب ويمرّر `defaultValue` إلى نموذج عميل.)
- أضف رفع الصورة إلى النموذج. انتبه: `Content-Type` يجب أن يكون `multipart/form-data` لا `application/json` — ستحتاج مسارًا مختلفًا في `api.ts`.
- عالج انتهاء الرمز: عندما يعيد الـ `API` رمز `401`، جرّب التجديد تلقائيًا بـ `refresh_token` ثم أعِد الطلب مرّة واحدة. أين المكان الصحيح لهذا المنطق؟
- أضف رسالة نجاح بعد الإنشاء (تلميح: معاملة استعلام، أو `cookie` مؤقتة).
- استبدل `confirm()` بحوار تأكيد حقيقي. لماذا `confirm` سيّئ في تطبيق حديث؟

الجاهزية والنشر

ما يفصل «يعمل على جهازي» عن «منشور ويعمل لغيري».

خطوة ٧١ وثّق الـ API بـ Swagger

توثيق تفاعلي يُولّد من الكود نفسه.

```
npm i @nestjs/swagger
```

```
api/src/main.ts
```

```
import { DocumentBuilder, SwaggerModule } from "@nestjs/swagger";

if (process.env.NODE_ENV !== "production") {
  const config = new DocumentBuilder()
    .setTitle("mini-shop API")
    .setDescription("Products, categories and auth")
    .setVersion("1.0")
    .addBearerAuth() // زوّج Authorize في الواجهة
    .build();

  const document = SwaggerModule.createDocument(app, config);
  SwaggerModule.setup("docs", app, document);
}
```

وزيّن الـ DTOs ليصير التوثيق مفيدًا فعليًا:

```
import { ApiProperty } from "@nestjs/swagger";

export class CreateProductDto {
  @ApiProperty({ example: "blue-mug", description: "URL-safe unique identifier" })
  slug!: string;

  @ApiProperty({ example: 12.5, minimum: 0 })
  price!: number;
}
```

جربها الآن

افتح <http://localhost:3000/docs>. ستري كل نقاط النهاية مصنّفة، ويمكنك تنفيذها من المتصفح مباشرة. اضغط `Authorize`، الصق رمز الوصول، ثم جرّب نقطة محمية — يجب أن تنجح. هذا الرابط وحده يستحق ذكره في سيرتك الذاتية: يريك المُقابل الـ `API` في ثلاثين ثانية بلا تنصيب.

[أغلق /docs في الإنتاج](#)

أو ضعه خلف مصادقة. توثيق كامل مكشوف يعطي المهاجم خريطة جاهزة لكل نقاط النهاية والحقول. لهذا وضعناه داخل شرط `.NODE_ENV !== "production"`.

خطوة ٧٢ سجلّات مفيدة

مع معرّف طلب يربط الأسطر ببعضها.

api/src/common/interceptors/logging.interceptor.ts

```
import { CallHandler, ExecutionContext, Injectable, Logger, NestInterceptor } from "@nestjs/common";
import { randomUUID } from "crypto";
import { tap } from "rxjs";

@Injectable()
export class LoggingInterceptor implements NestInterceptor {
  private readonly logger = new Logger("HTTP");

  intercept(ctx: ExecutionContext, next: CallHandler) {
    const req = ctx.switchToHttp().getRequest();
    const res = ctx.switchToHttp().getResponse();
    const id = randomUUID().slice(0, 8);
    const start = Date.now();

    res.setHeader("x-request-id", id); المستخدم يقدر يرسله لك عند شكواه

    return next.handle().pipe(
      tap(() => {
        const ms = Date.now() - start;
        this.logger.log(
          `[${id}] ${req.method} ${req.originalUrl} ${res.statusCode} ${ms}ms`,
        );
      }),
    );
  }
}
```

سجّله عالميًا: `app.useGlobalInterceptors(new LoggingInterceptor());`

ما لا يُسجّل أبدًا

كلمات السر، الرموز، أرقام البطاقات، بيانات شخصية كاملة. قاعدة عملية: لا تسجّل `req.body` كاملًا في الإنتاج — سجّل الحقول التي تحتاجها بالاسم. تسريب عبر السجّلات حادثة أمنية حقيقية.

خطوة ٧٣ حدّ المحاولات ورؤوس الأمان

```
npm i @nestjs/throttler helmet
```

```
ThrottlerModule.forRoot([ { ttl: 60_000, limit: 100 } ]), ١٠٠ طلب/دقيقة عامة

providers: [ { provide: APP_GUARD, useClass: ThrottlerGuard } ]
```

وشدّد على تسجيل الدخول تحديداً:

```
@Throttle({ default: { limit: 5, ttl: 60_000 } }) ٥ محاولات في الدقيقة
@Public()
@Post("login")
login(@Body() dto: LoginDto) { ... }
```

وفي `main.ts`: `app.use(helmet());`

جربها الآن

```
for i in $(seq 1 8); do
  curl -s -o /dev/null -w "%{http_code}\n" -X POST http://localhost:3000/api/auth/login \
    -H "Content-Type: application/json" -d '{"email":"a@b.c","password":"wrong"}'
done
```

المتوقع: خمسة 401 ثم 429 Too Many Requests. بهذا صار تخمين كلمات السر بالقوّة الغاشمة غير عملي.

خطوة ٧٤ فحص الصحة

كل منصة نشر تسأل عنه.

```
npm i @nestjs/terminus
```

```
@Public()
@Get("health")
@HealthCheck()
check() {
  return this.health.check([
    فحص حقيقي: هل قاعدة البيانات تستجيب؟
    () => this.prismaHealth.pingCheck("database", this.prisma),
  ]);
}
```

لماذا فحص القاعدة لا مجرد { status: "ok" } لأنّ التطبيق قد يكون حيّاً بينما قاعدة البيانات ساقطة — فيمّر الفحص ويستمرّ الموزّع في إرسال المستخدمين إلى نسخة معطوبة. الفحص المفيد يفحص التبعيات الحرجة.

```
api/src/products/products.service.spec.ts
```

```
import { Test } from "@nestjs/testing";
import { NotFoundException } from "@nestjs/common";
import { ProductsService } from "../products.service";
import { PrismaService } from "../../prisma/prisma.service";

describe("ProductsService", () => {
  let service: ProductsService;
  const prismaMock = {
    product: { findUnique: jest.fn(), count: jest.fn(), findMany: jest.fn() },
    $transaction: jest.fn(),
  };

  beforeEach(async () => {
    const moduleRef = await Test.createTestingModule({
      providers: [
        ProductsService,
        { provide: PrismaService, useValue: prismaMock }, استبدال التبعية
      ],
    }).compile();

    service = moduleRef.get(ProductsService);
    jest.clearAllMocks();
  });

  it("throws 404 when the product does not exist", async () => {
    prismaMock.product.findUnique.mockResolvedValue(null);
    await expect(service.findOne(42)).rejects.toThrow(NotFoundException);
  });

  it("returns the product when found", async () => {
    prismaMock.product.findUnique.mockResolvedValue({ id: 1, slug: "mug" });
    await expect(service.findOne(1)).resolves.toMatchObject({ slug: "mug" });
  });
});
```

```
npm test
```

لماذا الحقن يجعل الاختبار سهلاً لأنّ الخدمة لا تُنشئ تبعياتها بنفسها — تستقبلها. فتستطيع تمرير بديل في الاختبار بلا تعديل سطر واحد من كود الإنتاج. هذا هو الجواب العملي على «ما فائدة حقن التبعية؟» — أفضل بكثير من الجواب النظري.

api/test/auth.e2e-spec.ts

```
import * as request from "supertest";

describe("Auth (e2e)", () => {
  let app: INestApplication;
  let token: string;

  beforeAll(async () => {
    const moduleRef = await Test.createTestingModule({
      imports: [AppModule],
    }).compile();

    app = moduleRef.createNestApplication();
    app.setGlobalPrefix("api");
    app.useGlobalPipes(new ValidationPipe({ whitelist: true, transform: true }));
    await app.init();
  });

  afterAll(async () => app.close());

  it("rejects protected route without a token", () =>
    request(app.getHttpServer()).get("/api/auth/me").expect(401));

  it("logs in the seeded admin", async () => {
    const res = await request(app.getHttpServer())
      .post("/api/auth/login")
      .send({ email: "admin@minishop.test", password: "Admin@12345" })
      .expect(201);

    expect(res.body.accessToken).toBeDefined();
    token = res.body.accessToken;
  });

  it("returns the current user and never leaks the password", async () => {
    const res = await request(app.getHttpServer())
      .get("/api/auth/me")
      .set("Authorization", `Bearer ${token}`)
      .expect(200);

    expect(res.body.email).toBe("admin@minishop.test");
    expect(res.body.password).toBeUndefined(); تأكيد أمني في اختبار
  });
});
```

npm run test:e2e

اختبارات e2e تحتاج قاعدة بيانات منفصلة

وإلا ستمسح بيانات تطويرك. أنشئ `env.test`. بقاعدة `minishop_test`، وشغل الهجرات عليها قبل الاختبارات. القاعدة الذهبية: الاختبار لا يلمس بيانات حقيقية أبدًا.

خطوة ٧٧ احزم ال API في صورة Docker

بناء متعدّد المراحل: صورة نهائية صغيرة.

api/Dockerfile

```
FROM node:22-alpine AS builder
WORKDIR /app

COPY package*.json ./
RUN npm ci                    تمامًا lock يلتزم بـ — install لا ci

COPY prisma ./prisma
RUN npx prisma generate

COPY . .
RUN npm run build

--- مرحلة التشغيل: بلا أدوات بناء ولا تبعيات تطوير ---
FROM node:22-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production

COPY package*.json ./
RUN npm ci --omit=dev

COPY --from=builder /app/dist ./dist
COPY --from=builder /app/prisma ./prisma
COPY --from=builder /app/node_modules/.prisma ./node_modules/.prisma

EXPOSE 3000
شغل الهجرات ثم الخادم
CMD ["sh", "-c", "npx prisma migrate deploy && node dist/main.js"]
```

api/.dockerignore

```
node_modules
dist
.env
uploads
*.log
```

جربها الآن

```
docker build -t minishop-api ./api
docker images minishop-api
```

الحجم المتوقع حوالي 250-400MB. لو بنيت بمرحلة واحدة لتجاوز الغيغابايت. `npm ci --omit=dev` وحده يحذف مئات الميغابايتات.

خطوة ٧٨ انشر

ثلاث خدمات مجانية، وترتيب لا يُعكس.

الجزء	الخدمة	ملاحظات
قاعدة البيانات	Neon	Postgres مُدار، طبقة مجانية، انسخ <code>DATABASE_URL</code>
API ال	Railway	يبنى من <code>Dockerfile</code> تلقائيًا
الواجهة	Vercel	صانعة <code>Next</code> — الأسهل والأفضل

الترتيب مهم:

1. أنشئ قاعدة على Neon وانسخ رابط الاتصال.
2. ارفع المستودع إلى GitHub.
3. في Railway: مشروع جديد من المستودع، جذر الخدمة `/api`، وأضف المتغيرات: `DATABASE_URL`، `JWT_ACCESS_SECRET`، `JWT_REFRESH_SECRET`، `NODE_ENV=production`، `WEB_ORIGIN` (اتركه مؤقتًا ثم عُد إليه).
4. انسخ رابط ال API الناتج.
5. في Vercel: استورد المستودع، جذر المشروع `/web`، وأضف `NEXT_PUBLIC_API_URL` و `API_URL_INTERNAL` برابط Railway.
6. عُد إلى Railway واضبط `WEB_ORIGIN` على رابط Vercel — وإلا حجب CORS كل شيء.
7. شغل البذور مرّة واحدة على قاعدة الإنتاج (من طرفية Railway أو محليًا مع `DATABASE_URL` الإنتاجي).

أسرار الإنتاج ليست أسرار التطوير

ولّد أسرار JWT جديدة تمامًا للإنتاج:

```
node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"
```

استعمال نفس السرّ في البيئتين يعني أنّ رمزًا مُصدّرًا على جهازك صالح على الإنتاج.

وتكامل مستمر بسيط، ليُرفض أي دفع يكسر البناء:

```
name: CI
on: [push, pull_request]

jobs:
  api:
    runs-on: ubuntu-latest
    defaults: { run: { working-directory: api } }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: npm, cache-dependency-path: api/package-lock.json }
      - run: npm ci
      - run: npx prisma generate
      - run: npx tsc --noEmit      فحص الأنواع
      - run: npm run lint
      - run: npm test
```

تمرين المرحلة ١٠

1. أضف مهمة web إلى ملف الـ CI تشغّل `npm run build` على الواجهة.
2. أضف HEALTHCHECK إلى الـ `Dockerfile` يستدعي `./api/health`.
3. اقرأ سجلات Railway بعد أول نشر: هل نجحت `migrate deploy`؟ ابحث عن سطرها.
4. افتح موقعك المنشور على هاتفك بلغة عربية. هل الاتجاه صحيح؟ هل الصور تظهر؟
5. غيّر `WEB_ORIGIN` عمداً إلى رابط خاطئ وشاهد خطأ `CORS` في وحدة تحكم المتصفح. أعده. الآن تعرف شكل المشكلة حين تراها.

وصلت — ماذا الآن؟

قائمة تحقق، خطوات التوسيع، وأسئلة ستُسأل عن هذا المشروع بالذات.

قائمة التحقق النهائية

مرّ عليها بندًا بندًا. أي بند لا تستطيع إثباته بتجربة الآن، ارجع إلى خطوته.

المجال	البند
البيئة	قاعدة البيانات تعمل في حاوية وبياناتها تنجو من إعادة التشغيل
	<code>env</code> خارج <code>git</code> و <code>env.example</code> داخله
	التطبيق يرفض الإقلاع عند نقص متغيّر بيئة
قاعدة البيانات	المخطّط فيه علاقات وفهارس و <code>onDelete</code> صريح
	تاريخ الهجرات في <code>git</code> ، والبذور قابلة لإعادة التشغيل
	السعر <code>Decimal</code> لا <code>Float</code>
API ال	مورد <code>CRUD</code> كامل برموز حالة صحيحة (201/200/204/404/409)
	ترقيم وفلترة وبحث وترتيب، مع سقف صارم لحجم الصفحة
	كل جسم طلب يمزّ ب <code>DTO</code> مُتحقّق منه، والحقول الزائدة مرفوضة
	لا استعلام داخل حلقة في أي مكان
الأمان	كلمات السر مجرّاة، ولا تظهر في أي استجابة
	الحماية هي الافتراض، والفتح صريح ب <code>@Public()</code>
	الرمز في كوكبي <code>httpOnly</code> لا في <code>localStorage</code>
	حدّ محاولات على الدخول، ورسالة فشل واحدة لا تكشف شيئًا
اللغات	الطبقات الثلاث مفصولة، ولكلّ أدواتها
	احتياط عند غياب الترجمة — لا فراغ للمستخدم أبدًا
الواجهة	مكوّنات خادم افتراضيًا، و <code>"use client"</code> في أصغر نطاق ممكن
	حالة الفلاتر في الرابط، لا في <code>useState</code>
	<code>loading.tsx</code> و <code>error.tsx</code> و <code>not-found.tsx</code> موجودة
الجاهزية	توثيق <code>Swagger</code> مغلق في الإنتاج
	اختبارات وحدة و <code>e2e</code> تعمل في <code>CI</code>
	منشور فعليًا، والرابط يعمل من هاتف شخص آخر

ماذا تضيف بعد ذلك

مرتبة بحسب ما يضيفه كل بند إلى فرصتك في المقابلة:

الإضافة	ما تُثبته عنك
سَلّة وطلبات	معاملات أعقد، حالات (order status)، منطق أعمال حقيقي
رفع الصور إلى S3 أو R2	فهم أنّ نظام ملفات الحاوية زائل
بحث كامل النص بـ Postgres	معرفة بقاعدة البيانات تتجاوز الـ ORM
ذاكرة مؤقتة Redis لقائمة المنتجات	تفكير في الأداء وإبطال الذاكرة
طابور BullMQ لإرسال بريد الترحيب	فصل العمل البطيء عن الطلب — يقابل Queue في Laravel
تحديث فوري بـ WebSocket	Nest Gateways — يميّزك في وظائف كثيرة
تخزين مؤقت بـ TanStack Query	إدارة حالة الخادم في العميل
GitHub Actions ينشر تلقائيًا	CI/CD كاملاً

نصيحة عملية

مشروع واحد عميق منشور ومختبر أقوى بكثير من خمسة مشاريع ناقصة على GitHub. خذ mini-shop وأضف إليه، ولا تبدأ من الصفر كل مرّة. واكتب README جيّدًا: صورة للمشروع، طريقة التشغيل في ثلاثة أوامر، ورابط منشور — هذه أوّل ثلاث دقائق يقضيها المُقايِل مع عملك.

أسئلة مقابلة على هذا المشروع بالذات

لن يسألوك أسئلة نظرية مجرّدة إن كان أمامهم مشروعك؛ سيسألون عن قراراتك فيه. جهّز هذه:

البنية وال API

السؤال	جوهر الجواب
لماذا فصلت الواجهة عن ال API؟	عميل ثانٍ محتمل (موبايل)، فريقان بالتوازي، نشر مستقل، وعقد واضح بينهما
لماذا NestJS لا Express عاريًا؟	بنية مفروضة، حقن تبعيات يجعل الاختبار سهلًا، ووحدات تنمو بلا فوضى. Express يكفي لخدمة صغيرة
الفرق بين Guard و Interceptor و Pipe و Filter؟	الحارس يقرّر المرور، والأنبوب يحوّل ويتحقّق من المدخل، والمعتراض يلفّ التنفيذ (قبل وبعد)، والفلتر يلتقط الاستثناءات
لماذا PATCH لا PUT؟	PUT استبدال كامل، و PATCH تعديل جزئي — و DTO التعديل عندنا جزئي
كيف تعالج الأخطاء بشكل موحد؟	استثناءات Nest المدمجة + فلتر يترجم أخطاء Prisma المعروفة، ورسالة عامّة لأي شيء غير متوقّع

قاعدة البيانات

السؤال	جوهر الجواب
كيف تتجنّب N+1؟	<code>include</code> أو <code>select</code> بدل الاستعلام داخل حلقة؛ والعلامة التحذيرية هي <code>await</code> داخل <code>for</code>
كيف تضيف عمودًا مطلوبًا إلى جدول فيه بيانات؟	ثلاث هجرات: اختياري → ملء البيانات → جعله مطلوبًا. بلا فقدان بيانات
متى تستعمل معاملة؟	عند كل كتابة تمسّ أكثر من جدول ويجب أن تنجح كوحدة واحدة
لماذا جدول ترجمات لا أعمدة لكل لغة؟	إضافة لغة تصير صفوفًا لا هجرة على كل جدول؛ الثمن <code>join</code> إضافي — مقايضة واعية
<code>migrate dev</code> أم <code>migrate deploy</code> ؟	الأول للتطوير (يوّلد وقد يعيد البناء)، والثاني للإنتاج (ينقذ الموجود فقط)

المصادقة والأمان

السؤال	جوهر الجواب
لماذا رمز وصول ورمز تجديد؟	رمز قصير يقلّل نافذة الضرر عند السرقة، ورمز طويل مخزّن في القاعدة يمكن إبطاله فورًا
أين تخزّن الرمز في المتصفّح ولماذا؟	كوكي <code>httpOnly + SameSite=Lax</code> . <code>localStorage</code> مكشوف لأي XSS
كيف تُسجّل خروجًا فورًا مع أنّ JWT عديم الحالة؟	الإبطال يقع على رمز التجديد المخزّن؛ ورمز الوصول قصير جدًا فينتهي وحده
401 أم 403؟	الأول «لا أعرفك»، والثاني «أعرفك ولا يُسمح لك»
لماذا نفس الرسالة للبريد الخاطئ ولكلمة السر الخاطئة؟	منع تعداد المستخدمين
هل الـ <code>middleware</code> في <code>Next</code> يكفي للحماية؟	لا — يفحص وجود كوكي فقط. الأمان في الـ <code>API</code> وفي فحص مكوّن الخادم

React و Next.js

السؤال	جوهر الجواب
مكوّن خادم أم عميل؟ كيف تقرّر؟	خادم افتراضيًا؛ انزل إلى العميل عند الحاجة إلى حالة أو حدث أو واجهة متصفّح — وفي أصغر نطاق
SSR و SSG و ISR: الفرق ومتى؟	<code>no-store</code> لكل طلب · بلا خيار = وقت البناء · <code>n: revalidate</code> وسط بينهما
ما <code>Server Action</code> ولماذا استعملتها للدخول؟	دالة تعمل على الخادم تُستدعى من نموذج؛ كلمة السر والرمز لا يمرّان بالمتصفّح، ويعمل النموذج قبل تحميل JS
لماذا الفلاتر في الرابط لا في <code>useState</code> ؟	قابلية المشاركة، وزرّ الرجوع، وإعادة تصيير الخادم بالبيانات الصحيحة
ما <code>revalidatePath</code> ؟	إبطال الذاكرة المؤقتة بعد الكتابة؛ نسيانه يعني بيانات قديمة على الشاشة
ما خطر <code>NEXT_PUBLIC</code> ؟	يُحقن في حزمة المتصفّح ويراه الجميع — للعناوين العامة فقط

السؤال	جوهر الجواب
كيف تدعم لغتين في تطبيق كامل؟	ثلاث طبقات مفصولة: نصوص الواجهة (next-intl)، رسائل الـ API (nextjs-i18n)، محتوى القاعدة (جدول ترجمات)
كيف يعرف الخادم لغة الطلب؟	ترويسة Accept-Language، مع بادئة اللغة في المسار كمصدر أوثق في الواجهة
ماذا لو نقصت ترجمة؟	سلسلة احتياط تنتهي بقيمة قابلة للعرض دائمًا
كيف تدعم RTL بلا تكرار CSS؟	dir على <html> + خصائص منطقية (ms-* / margin-inline-start) بدل يمين ويسار

والعمل مع الـ AI

ذكرت أنّ الشغل اليومي سيكون بمساعدة الذكاء الاصطناعي. هذا واقع أغلب الفرق اليوم، ولا عيب فيه — لكن المقابلة تقيس شيئًا مختلفًا عمّا يقيسه الشغل اليومي. ثلاث قواعد تحفظ لك الاثنين:

- اكتب كل مفهوم بيدك أول مرّة. بعد ذلك ولّده كما تشاء. ما لم تكتبه مرّة واحدة لن تستطيع الدفاع عنه على السبّورة.
- لا تدمج كودًا لا تستطيع شرحه سطرًا سطرًا. اسأل «لماذا هنا؟ وما البديل؟» حتى تفهم، ثم ادمج. الكود الذي لا تفهمه دين تدفعه وقت العطل.
- المعمارية قرارك أنت. أين يقع المنطق، وشكل البيانات، وحدود الوحدات — هذه القرارات هي عملك، والأداة تنقّذ. وهي بالضبط ما يُسأل عنه في المقابلة.

آخر كلمة ما بنيته هنا ليس تمرينًا: هو مشروع بمصادقة حقيقية وAPI كامل ولغتين واختبارات ونشر. أغلب من يتقدّم لهذه الوظائف لا يملك واحدًا مثله. شغّله، اكسره، أصلحه، ثم أضف إليه ميزة كل أسبوع. وحين تُسأل «حدّثني عن مشروع بنيته»، سيكون عندك جواب مدّته عشر دقائق — مدعوم برابط يعمل.

بالتوفيق يا يزن.